Dynamic Programming Templates

Six template families. Every problem maps to one loop skeleton and one dp-state definition.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
70	Climbing Stairs	Each step you can climb 1 or 2 steps. How many distinct ways to reach step n?	the last move was a 1-step or a 2-step, so ways to reach n = ways to n-1 + ways to n-2.	O(n)	O(n)	Standard
198	House Robber	Rob houses on a line — can't rob two adjacent. Maximize total.	at each house, either skip it (keep `dp[i-1]`) or rob it (`dp[i-2] + value`, since i-1 is now off-limits).	O(n)	O(n)	Standard
213	House Robber II	Houses arranged in a circle — first and last are adjacent. Can't rob both.	breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.	O(n)	O(1)	Standard
300	Longest Increasing Subsequence	Length of the longest strictly increasing subsequence.	the best subsequence ending at i extends the longest earlier one whose last value is still smaller.	$O(n^2)$	O(n)	Standard
139	Word Break	Can `s` be segmented into a sequence of dictionary words?	a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.	$O(n^2 \cdot L)$ where L = substring length	O(n)	Standard
416	Partition Equal Subset Sum	Can the array be partitioned into two subsets with equal sum?		O(n)	O(sum)	Standard
494	Target Sum	Assign `+` or `-` to each number to reach `target`. How many ways?		O(n)	O(sum)	Standard
474	Ones and Zeroes	Given strings of '0'/'1', find the largest subset using at most `m` zeros and `n` ones.		O(l)	O(mn)	Standard
322	Coin Change	Minimum number of coins to make `amount`. Coins can be reused.		O(amount)	O(amount)	Standard
518	Coin Change II	Number of distinct combinations (order doesn't matter) to make `amount`.		O(amount)	O(amount)	Standard
377	Combination Sum IV	Number of ordered sequences (order matters) that sum to `target`.		O(target)	O(target)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1143	Longest Common Subsequence	Length of the longest subsequence present in both strings.	if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.	O(m·n)	O(m·n)	Standard
72	Edit Distance	Minimum operations (insert, delete, replace) to convert `word1` to `word2`.	matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.	O(m·n)	O(m·n)	Standard
115	Distinct Subsequences	Number of ways `s` contains `t` as a subsequence.	you can always skip the current char of s; if it matches the current char of t, you may also use it to advance t.	O(m·n)	O(m·n)	Standard
647	Palindromic Substrings	Count all palindromic substrings of `s`.	`s[i..j]` is a palindrome when its two ends match AND the inside `s[i+1..j-1]` already is.	$O(n^2)$	$O(n^2)$	Standard
516	Longest Palindromic Subsequence	Length of the longest subsequence of `s` that is a palindrome.	matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.	$O(n^2)$	$O(n^2)$	Standard
312	Burst Balloons	Burst all balloons to maximize coins. Coins from bursting balloon `k` between open boundaries `i` and `j` = `balloons[i] * balloons[k] * balloons[j]`.		$O(n^3)$	$O(n^2)$	Standard
62	Unique Paths	Number of paths from top-left to bottom-right, moving only right or down.	you reach a cell only from above or from the left, so its path count is the sum of those two.	O(m·n)	O(m·n)	Standard
64	Minimum Path Sum	Minimum sum path from top-left to bottom-right, moving only right or down.	the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.	O(m·n)	O(m·n)	Standard
221	Maximal Square	Find the largest all-1 square in a binary matrix. Return its area.		O(m·n)	O(m·n)	Standard
329	Longest Increasing Path in a Matrix	Find the longest strictly increasing path. Can move in all 4 directions.	strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.	O(m·n)	O(m·n)	Standard
121	Best Time to Buy and Sell Stock (at most 1 transaction)		`hold` is the best profit if you currently own a share bought at the lowest price seen;	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			sell whenever today's price beats that.			
122	Best Time to Buy and Sell Stock II (unlimited transactions)		with unlimited trades, buying can fund itself from accumulated `cash`, so capture every upward move.	O(n)	O(1)	Standard
123	Best Time to Buy and Sell Stock III (at most 2 transactions)		the second buy can only spend profit left after the first sell, so chain the states in trade order.	O(n)	O(1)	Standard
309	Best Time to Buy and Sell Stock with Cooldown	After selling, must wait 1 day before buying again.	the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.	O(n)	O(1)	Standard
714	Best Time to Buy and Sell Stock with Transaction Fee	Pay a fee per transaction (on sell). Unlimited transactions.	the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.	O(n)	O(1)	Standard
746	Min Cost Climbing Stairs	Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).		O(n)	O(n)	Standard
91	Decode Ways	A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.		O(n)	O(n)	Two sources at each position. If the current digit is non-zero, it can be decoded alone (`dp[i] += dp[i−1]`). If the previous two digits form a valid two-letter code (10-26), add `dp[i−2]` .
279	Perfect Squares	Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to `n` .		O(n√n)	O(n)	iterate all squares ≤ i
96	Unique Binary Search Trees	Return the number of structurally unique BSTs that store values 1 to n.		$O(n^2)$	O(n)	Catalan number recurrence. `dp[i]` = number of unique BSTs with i nodes. When root = j, left subtree has j-1 nodes, right subtree has i-j nodes: `dp[i] += dp[j−1] * dp[i−j]` .

#	Name	Description	Intuition	Time	Space	Key Implementation
63	Unique Paths II	A robot moves from top-left to bottom-right in an m×n grid with obstacles. Count unique paths. Cells with `obstacleGrid[i][j] == 1` are blocked.		O(m·n)	O(m·n)	Same as #62 (Unique Paths) but skip cells with obstacles: `dp[i][j] = 0` if blocked, else `dp[i−1][j] + dp[i][j−1]`.
583	Delete Operation for Two Strings	Return the minimum number of deletions to make `word1` and `word2` equal.		O(m·n)	O(m·n)	Reduce to LCS. Minimum deletions = `len(word1) + len(word2) − 2 * LCS(word1, word2)`. Compute LCS using standard 2D DP.
368	Largest Divisible Subset	Find the largest subset of distinct positive integers such that for every pair (a, b), either `a % b == 0` or `b % a == 0`.		O(n^2)	O(n)	Sort first, then apply LIS-style DP. `dp[i]` = size of largest divisible subset ending at `nums[i]`. If `nums[i] % nums[j] == 0`, then `dp[i] = max(dp[i], dp[j] + 1)`. Track parent indices to reconstruct the subset.
931	Minimum Falling Path Sum	Given an n×n matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.		O(n^2)	O(n^2)	Grid DP with three sources from the row above. `dp[i][j] = matrix[i][j] + min(dp[i−1][j−1], dp[i−1][j], dp[i−1][j+1])`.
10	Regular Expression Matching	Implement regex matching with `.` (any single char) and `*` (zero or more of the preceding element). Match must cover the entire input string.		O(m·n)	O(m·n)	2D DP. `dp[i][j]` = whether `s[0..i)` matches `p[0..j)`. The `*` has two cases: zero occurrences (`dp[i][j−2]`) or one-more occurrence when the preceding pattern char matches `s[i−1]`.
87	Scramble String	Given s1 and s2, return true if s2 is a scrambled string of s1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them).		O(n^4)	O(n^3)	memoized recursion over (i1, i2, len). At each length, try every split point both swapped and non-swapped.
1049	Last Stone Weight II	Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone.		O(n·sum)	O(sum)	equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum ≤ total/2.
1312	Minimum Insertion Steps to Make a	Return the minimum number of insertions		O(n^2)	O(n^2)	answer = n − longest palindromic subsequence (LPS). LPS = LCS of the

#	Name	Description	Intuition	Time	Space	Key Implementation
	String Palindrome	needed to make a string a palindrome.				string and its reverse; or solve directly via interval DP.
44	Wildcard Matching	Match string `s` against pattern `p` with `?` (any single char) and `*` (any sequence including empty).	`*` either consumes one more char of `s` (`dp[i−1][j]`) or matches empty (`dp[i][j−1]`); everything else is a 1-to-1 char/ `?` match.	O(m·n)	O(m·n)	'*' matches empty prefix
53	Maximum Subarray	Find the contiguous subarray with the largest sum.	the best sum ending at `i` either extends the previous best or restarts at `nums[i]` (Kadane).	O(n)	O(1)	Kadane restart vs extend
97	Interleaving String	Return whether `s3` is formed by interleaving `s1` and `s2` while preserving each one's order.	the last char of `s3[0..i+j)` comes from either `s1` or `s2` ; reuse the answer for the smaller prefix.	O(m·n)	O(m·n)	take from s1
120	Triangle	Find the minimum path sum from top to bottom, moving to adjacent indices on the row below.	working bottom-up, each cell's best is its value plus the cheaper of the two children directly below.	$O(n^2)$	O(n)	bottom-up over rows
132	Palindrome Partitioning II	Return the minimum number of cuts so that every substring of the partition is a palindrome.	if `s[j..i]` is a palindrome, a cut after `j−1` lets `cuts[i] = cuts[j−1] + 1` ; precompute palindromes with interval DP.	$O(n^2)$	$O(n^2)$	interval DP precompute palindromes
140	Word Break II	Return all sentences obtained by segmenting `s` into space-separated dictionary words.	memoize on each suffix the list of valid segmentations, prepending each matching word to the segmentations of the remainder.	$O(n^2 \cdot 2^n)$ worst case	$O(2^n)$	memoized DFS returns sentences
152	Maximum Product Subarray	Find the contiguous subarray with the largest product.	a negative number flips max and min, so track both; the running max product ending at `i` may come from the prior max or min.	O(n)	O(1)	negative swaps max/min
174	Dungeon Game	Find the minimum starting health so the knight survives from top-left to bottom-right (each cell adds/subtracts health, must stay ≥ 1).	health needed depends on the future, so fill from the bottom-right: needed entering a cell = max(1, min child need − cell value).	O(m·n)	O(m·n)	fill from bottom-right

#	Name	Description	Intuition	Time	Space	Key Implementation
188	Best Time to Buy and Sell Stock IV	Maximize profit with at most `k` transactions.	for each allowed transaction count `t`, track best buy and sell; this is #123 generalized to `k` chained state pairs.	O(n·k)	O(k)	k chained state pairs
264	Ugly Number II	Find the nth ugly number (positive integers whose only prime factors are 2, 3, 5).	every ugly number is a previous ugly number times 2, 3, or 5; merge the three multiples with pointers.	O(n)	O(n)	merge multiples via pointers
313	Super Ugly Number	Find the nth super ugly number whose prime factors all come from a given `primes` list.	generalize #264 to arbitrary primes — keep one pointer per prime and pick the minimum next candidate.	O(n·k)	O(n+k)	one pointer per prime
337	House Robber III	Houses form a binary tree; you cannot rob two directly-linked houses. Maximize the total.	each node returns two values — best if it is robbed (skip children) vs not robbed (children free to choose).	O(n)	O(h)	tree DP returns {notRob, rob}
343	Integer Break	Break `n` into the sum of at least two positive integers maximizing their product.	for each split `j`, the rest can stay as `i-j` or be broken further (`dp[i-j]`); take the best.	O(n²)	O(n)	split point, break further or not
375	Guess Number Higher or Lower II	Find the minimum amount of money to guarantee a win when guessing a number in `[1, n]`, paying the guessed value on each wrong guess.	for range `[i,j]`, guessing `k` costs `k` plus the worst of the two resulting subranges; minimize over `k`.	O(n³)	O(n²)	interval DP over range length
376	Wiggle Subsequence	Find the length of the longest subsequence whose consecutive differences strictly alternate between positive and negative.	track the best subsequence ending with an up-move and with a down-move; each rise extends `down`, each fall extends `up`.	O(n)	O(1)	rise extends down-ending seq
413	Arithmetic Slices	Count the number of arithmetic subarrays (length ≥ 3, constant difference).	if `nums[i]` continues the arithmetic run ending at `i-1`, it adds `dp[i-1]+1` new slices ending at `i`.	O(n)	O(1)	extend arithmetic run
446	Arithmetic Slices II - Subsequence	Count the arithmetic subsequences (length ≥ 3) of the array.	for each pair `(j, i)` with difference `d`, weak slices ending at `i` extend those ending at `j`; promote weak to valid when length reaches 3.	O(n²)	O(n²)	per-index map keyed by difference
486	Predict the Winner	Two players alternately take a number from either end; decide whether	on range `[i,j]` the current player maximizes value –	O(n²)	O(n²)	interval DP, score difference

#	Name	Description	Intuition	Time	Space	Key Implementation
		player 1 can win (score ≥ player 2).	opponent's best on the remaining range.			
509	Fibonacci Number	Return the nth Fibonacci number.	each term is the sum of the previous two — the canonical linear 1D recurrence.	O(n)	O(1)	Standard
552	Student Attendance Record II	Count length-`n` attendance records that are rewardable (at most one 'A', no three consecutive 'L'), modulo 1e9+7.	state = (number of 'A's so far 0/1, trailing 'L' run 0/1/2); each day append P, A, or L respecting limits.	O(n)	O(1)	state = (A count, trailing L run)
576	Out of Boundary Paths	Count paths that move a ball off the grid in exactly `maxMove` steps from a start cell, modulo 1e9+7.	moving off the boundary counts as one path; otherwise spread current-step counts to 4 neighbors for the next step.	O(maxMove·m·n)	O(m·n)	spread to 4 neighbors
600	Non-negative Integers without Consecutive Ones	Count integers in `[0, n]` whose binary representation has no two adjacent 1 bits.	scan bits high-to-low; precompute Fibonacci-style counts of valid suffixes, adding them when a free bit is `1`, and stop early if two consecutive 1s appear in `n`.	O(1) (32 bits)	O(1)	fib[i] = valid i-bit strings
629	K Inverse Pairs Array	Count permutations of `1..n` with exactly `k` inverse pairs, modulo 1e9+7.	inserting `n` into a permutation of `1..n−1` adds `0..n−1` inversions; this gives a sliding-window sum over the previous row.	O(n·k)	O(n·k)	prefix sums of previous row
638	Shopping Offers	Buy exactly `needs[i]` of each item at minimum cost, given unit prices and special bundle offers (each usable multiple times).	treat the remaining needs vector as the state; either buy everything at unit price, or apply an affordable offer and recurse.	O(offers·$\prod$needs$_i$)	O($\prod$needs$_i$)	DFS+memo over needs vector
639	Decode Ways II	Count decodings of a digit string where `*` is a wildcard for digits 1-9, modulo 1e9+7.	extend #91 — each position can decode one char (count 1-9 options) or two chars (count valid 10-26 combinations); `*` multiplies the options.	O(n)	O(n)	single char, count options
650	2 Keys Keyboard	Starting with one 'A', using only Copy-All and Paste, return the minimum operations to obtain `n` 'A's.	the answer is the sum of `n` 's prime factors — building `i` from a divisor `j` costs `i/j` operations.	O(n$\sqrt{n}$)	O(n)	build i from divisor j
673	Number of Longest Increasing Subsequence	Count the number of longest strictly increasing subsequences.	alongside LIS length, track how many ways achieve it; a longer extension resets the count, an equal-length extension accumulates it.	O(n^2)	O(n)	longer extension resets count

#	Name	Description	Intuition	Time	Space	Key Implementation
674	Longest Continuous Increasing Subsequence	Find the length of the longest continuous (contiguous) strictly increasing subarray.	extend the current run while values rise, otherwise restart; a simpler linear scan of #300.	O(n)	O(1)	contiguous run, reset on drop
688	Knight Probability in Chessboard	Return the probability that a knight remains on an `n×n` board after exactly `k` moves from a start cell, each move chosen uniformly among 8.	spread the probability at each cell to its 8 knight-move targets per step; off-board moves lose probability.	O(k·n²)	O(n²)	8 knight moves, each prob 1/8
718	Maximum Length of Repeated Subarray	Find the length of the longest subarray common to two arrays (contiguous).	unlike LCS, the match must be contiguous, so a mismatch resets the running length to 0.	O(m·n)	O(m·n)	contiguous, no carry on mismatch
740	Delete and Earn	Deleting `nums[i]` earns its value but removes all copies of `nums[i]−1` and `nums[i]+1`; maximize earnings.	bucket points by value, then it becomes House Robber over the value line — you cannot take adjacent values.	O(n + max)	O(max)	bucket points by value
873	Length of Longest Fibonacci Subsequence	Given a strictly increasing array, find the length of the longest Fibonacci-like subsequence (each term = sum of the previous two).	a fib-like subseq is determined by its last two elements `(j, i)`; look up whether the required predecessor `arr[i]−arr[j]` exists earlier.	O(n²)	O(n²)	predecessor before arr[j]
887	Super Egg Drop	With `k` eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case.	invert the problem — `dp[m][k]` = highest floor count solvable with `m` moves and `k` eggs; a move either breaks (test below) or survives (test above), plus the current floor.	O(k·log n)	O(n·k)	dp[m][k] = floors solvable in m moves
918	Maximum Sum Circular Subarray	Find the maximum subarray sum where the array is circular (subarray may wrap around).	the answer is either a normal max subarray (Kadane) or the total minus the minimum subarray (wrap); guard the all-negative case.	O(n)	O(1)	track min subarray for wrap
935	Knight Dialer	Count distinct phone numbers of length `n` dialable by knight moves on a phone keypad, modulo 1e9+7.	from each digit a knight can reach a fixed set of digits; spread counts across the adjacency map each step.	O(n)	O(1)	knight adjacency per digit
983	Minimum Cost For Tickets	Given travel days and the cost of 1-day, 7-day, and 30-day passes, return the minimum cost to cover all travel days.	on a travel day choose the cheapest pass covering it by looking back 1, 7, or 30 days; non-travel days inherit the prior cost.	O(lastDay)	O(lastDay)	non-travel day inherits cost
1027	Longest Arithmetic	Find the length of the longest arithmetic	for each pair `(j, i)`, the arithmetic subseq	O(n²)	O(n²)	per-index map keyed by difference

#	Name	Description	Intuition	Time	Space	Key Implementation
	Subsequence	subsequence (constant difference).	ending at `i` with difference `d` extends the one ending at `j`.			
1048	Longest String Chain	Find the longest chain of words where each word is a predecessor of the next (insert exactly one char).	sort by length; for each word, try removing each character to find a shorter predecessor and extend its best chain.	$O(n \cdot L^2)$	O(n)	process shortest first
1137	N-th Tribonacci Number	Return the nth Tribonacci number (each term is the sum of the previous three).	the linear 1D recurrence with three rolling variables instead of two.	O(n)	O(1)	sum of previous three
1216	Valid Palindrome III	Return whether `s` becomes a palindrome after removing at most `k` characters.	the minimum removals to make `s` a palindrome equals `n - LPS(s)`; check whether that is $\le$ `k`.	$O(n^2)$	$O(n^2)$	interval DP for LPS
1218	Longest Arithmetic Subsequence of Given Difference	Find the longest arithmetic subsequence with a fixed difference `difference`.	with a known difference, the predecessor of `num` must be `num - difference`; one hashmap pass suffices.	O(n)	O(n)	fixed difference, value-keyed map
1269	Number of Ways to Stay in the Same Place After Some Steps	Count the ways to return to index 0 after `steps` moves (stay, left, or right) without leaving an array of size `arrLen`, modulo 1e9+7.	position can never exceed `steps/2`, so cap the reachable range; each step a position spreads to stay/left/right.	O(steps·min(arrLen, steps))	O(min(arrLen, steps))	cap reachable positions
1395	Count Number of Teams	Count triplets `(i, j, k)` with `i < j < k` whose ratings are strictly increasing or strictly decreasing.	fix the middle soldier `j`; the answer is its number of smaller-before × larger-after, plus the mirror case.	$O(n^2)$	O(1)	fix middle soldier
1567	Maximum Length of Subarray With Positive Product	Find the length of the longest subarray whose product is positive.	track the lengths of positive-product and negative-product runs ending at `i`; a negative number swaps them, a zero resets both.	O(n)	O(1)	zero resets both runs
1696	Jump Game VI	From index 0, each move jumps 1 to `k` indices forward; maximize the sum of values landed on, reaching the last index.	`dp[i]` = best score reaching `i` = `nums[i] +` max `dp[j]` in the window `[i-k, i-1]`; a monotonic queue keeps that window max in O(1).	O(n)	O(n)	monotonic queue of indices
1884	Egg Drop With 2 Eggs and N Floors	With exactly 2 eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case.	`dp[i]` = min worst-case moves for `i` floors; dropping at floor `j` costs `1 + max(dp[j-1] from`	$O(n^2)$	O(n)	first drop at floor j

#	Name	Description	Intuition	Time	Space	Key Implementation
			`break, dp[i-j] from survive)` .			

All Six Templates

Variables: `dp[i]` = the answer for the subproblem at index/state `i` (meaning varies per family: ways/cost/length/feasibility ending at or reaching `i`) · `n` = problem size · `target` / `j` = knapsack capacity dimension.
Pseudocode:

```
LINEAR 1D:   dp[i] = fixed recipe over dp[i-1], dp[i-2] (Kadane: best ending at i = max(start fresh, exte
LOOK-BACK:   for each i, scan all earlier j and extend the best valid dp[j]
KNAPSACK:    collapse item dimension to 1D; descending j = each item once, ascending j = reusable
2D SEQUENCE: match consumes both strings (diagonal), mismatch drops one side
INTERVAL:    solve short ranges first, combine via a split point inside the range
GRID:        each cell accumulates from cells it could arrive from (up / left)
STATE MACHINE: name a few states, update each from yesterday's states every step
```

```java
// 1. LINEAR 1D — each cell depends on a fixed number of previous cells
// the answer at i is a fixed recipe over the last one or two answers.  — WHEN: "ways/cost to reach ste
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);
// KADANE (max subarray, #53) = Linear-1D where dp[i] = best sum ENDING at i:
//   dp[i] = max(nums[i], dp[i-1] + nums[i])   // start fresh vs extend (drop a negative prefix)
//   answer = max over all dp[i]; collapse dp[] to one rolling int → O(1) space.
//   variants: #152 product (track max AND min, a negative swaps them); #918 circular.

// 2A. LOOK-BACK 1D — dp[i] depends on all j < i
// to finish position i, scan every earlier position j and extend the best valid one.  — WHEN: "longest
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]);        // e.g., LIS, word break
    }
}

// 2B/2C. KNAPSACK — collapse the item dimension to 1D; loop direction picks reuse.
//   0/1 (each item once): j DESCENDING  → reads previous row.
//   unbounded (reusable): j ASCENDING   → reads current row.
//   Mnemonic: DESCENDING = Distinct, ASCENDING = Again. Full code + permutation
//   variant + "why" in Part 2 — Knapsack DP.

// 3. 2D SEQUENCE — match consumes both, mismatch drops one side. WHEN: LCS / edit
//    distance / subsequence count over two strings. Full code → Part 3.
// 4. INTERVAL DP — short ranges first (i right-to-left, j from i+1; dp[i+1][*] ready).
//    WHEN: palindrome / merge-burst / split-point in a range. Full code → Part 4.
// 5. GRID DP — each cell accumulates from cells it could arrive from (up/left).
//    WHEN: grid paths / min-cost / largest square. Full code → Part 5.
// 6. STATE MACHINE — named states updated from yesterday's states each step.
//    WHEN: buy/sell/hold, limited transactions, cooldown/fee. Full code → Part 6.
```

Knapsack Decision Tree

```
Want count or min/max?
├── Count (dp[j] += dp[j - num])
│   ├── 0/1 (each item once):    j descending → #416, #494
│   └── Unbounded (reuse):       j ascending  → #518
│       └── Ordered (permuts):   target outer, nums inner → #377
└── Min/Max (dp[j] = min/max(...))
    ├── 0/1 minimize:            j descending → #474 (maximize)
    └── Unbounded minimize:      j ascending  → #322
```

Part 1 — Linear 1D DP

#70 Climbing Stairs

Description: Each step you can climb 1 or 2 steps. How many distinct ways to reach step n?
Intuition: the last move was a 1-step or a 2-step, so ways to reach n = ways to n-1 + ways to n-2.

Variables: `dp[i]` = number of distinct ways to reach step `i`. **Pseudocode:**

```
if n <= 2: return n
dp[1] = 1; dp[2] = 2
for i from 3 to n:
    dp[i] = dp[i-1] + dp[i-2]   // arrive via a 1-step or a 2-step
return dp[n]
```

```java
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int[] dp = new int[n + 1];
        dp[1] = 1; dp[2] = 2;
        for (int i = 3; i <= n; i++)
            dp[i] = dp[i-1] + dp[i-2];   // arrive via a 1-step or a 2-step
        return dp[n];
    }
}
```

Time O(n) | **Space** O(n)

#198 House Robber

Description: Rob houses on a line — can't rob two adjacent. Maximize total.
Intuition: at each house, either skip it (keep `dp[i-1]`) or rob it (`dp[i-2] + value`, since i-1 is now off-limits).

Variables: `dp[i]` = max money robbable from houses `0..i`; `dp[0]` = first house, `dp[1]` = better of first two.
Pseudocode:

```
n = number of houses
if only one house: return its value
dp[0] = nums[0]
dp[1] = max(nums[0], nums[1])
for i from 2 to n-1:
    dp[i] = max(dp[i-1],            // skip house i, keep best up to i-1
                dp[i-2] + nums[i])  // rob house i, add best up to i-2
return dp[n-1]                       // best over all houses
```

```java
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0]; dp[1] = Math.max(nums[0], nums[1]);
        for (int i = 2; i < n; i++)
            dp[i] = Math.max(dp[i-1], dp[i-2] + nums[i]);   // skip i, or rob i + best up to i-2
        return dp[n-1];
    }
}
```

Time O(n) | **Space** O(n)

#213 House Robber II

Description: Houses arranged in a circle — first and last are adjacent. Can't rob both.
Key: run House Robber on `[0, n-2]` and `[1, n-1]`; take the max. Two passes, exact same helper.
Intuition: breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.

Variables: in the helper, `prev1` = best rob ending at the previous index (`dp[k-1]`), `prev2` = best two indices back (`dp[k-2]`), `current` = `dp[k]`; two passes over ranges `[0,n-2]` and `[1,n-1]`. **Pseudocode:**

```
n = number of houses
if only one house: return its value
return max( robLine(0, n-2),   // exclude last house
            robLine(1, n-1) )  // exclude first house
robLine(i, j):
    prev2 = 0, prev1 = 0
    for k from i to j:
        current = max(prev1,            // skip house k
                      prev2 + nums[k])  // rob house k
        prev2 = prev1                    // slide window forward
        prev1 = current
    return prev1                         // best over the line
```

```java
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        return Math.max(rob(nums, 0, n - 2), rob(nums, 1, n - 1));
    }
    private int rob(int[] nums, int i, int j) {
        int prev2 = 0, prev1 = 0;
        for (int k = i; k <= j; k++) {
            int current = Math.max(prev1, prev2 + nums[k]);
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }
}
```

Time O(n) | **Space** O(1)

#300 Longest Increasing Subsequence

Description: Length of the longest strictly increasing subsequence.
Template: Look-back 1D — `dp[i]` depends on all `dp[j]` where `j < i` and `nums[j] < nums[i]`.
Intuition: the best subsequence ending at i extends the longest earlier one whose last value is still smaller.

Variables: `dp[i]` = length of the longest strictly increasing subsequence that *ends* at index `i`; `max` = best `dp[i]` seen so far. **Pseudocode:**

```
n = length of nums
dp[i] = 1 for all i        // each element alone is an LIS of length 1
max = 1
for i from 1 to n-1:
    for j from 0 to i-1:
        if nums[j] < nums[i]:                  // can extend subsequence ending at j
            dp[i] = max(dp[i], dp[j] + 1)
    max = max(max, dp[i])                       // track global best
return max
```

```java
class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int max = 1;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);  // extend a smaller-ending
            }
            max = Math.max(max, dp[i]);
        }
        return max;
    }
}
```

Time O(n^2) | **Space** O(n)

#139 Word Break

Description: Can `s` be segmented into a sequence of dictionary words?
Template: Look-back 1D — `dp[i]` is true if some `dp[j]` is true and `s[j..i)` is a word.
Intuition: a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.

Variables: `dp[i]` = true if the prefix `s[0..i)` (first `i` chars) can be segmented into dictionary words; `dp[0]` = true (empty prefix); `words` = the dictionary as a hash set for O(1) lookup. **Pseudocode:**

```
words = set of dictionary words
n = length of s
dp[i] = false for all i
dp[0] = true                                  // empty prefix is breakable
for i from 1 to n:
    for j from 0 to i−1:
        if dp[j] and words contains s[j..i):   // breakable cut + word tail
            dp[i] = true
            break                             // one valid split is enough
return dp[n]                                   // whole string breakable?
```

```java
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> words = new HashSet<>(wordDict);
        int n = s.length();
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && words.contains(s.substring(j, i))) {  // breakable cut + word tail
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }
}
```

Time O(n²·L) where L = substring length | **Space** O(n)

---

198 vs 300 vs 139 — Look-back patterns

	#198 House Robber	#300 LIS	#139 Word Break
dp[i] means:	max rob ending at i	LIS ending at i	can break s[0..i)
j range:	j = i−1, i−2 (fixed)	j = 0..i−1 (all)	j = 0..i−1 (all)
condition on j:	none (always)	nums[j] < nums[i]	dp[j] && word in dict
update:	max(dp[i−1], dp[i−2]+x)	max(dp[i], dp[j]+1)	dp[i] = true; break
answer:	dp[n−1]	max(dp)	dp[n]

Part 2 — Knapsack DP

Canonical Templates

Variables: `dp[j]` = number of ways to reach sum `j` (count flavor; `dp[0]=1` is the empty selection); `i` = item index, `j` = current capacity/target, `num` = an item's weight/value. Loop *direction* decides reuse: descending j = item used at most once, ascending j = item reusable; target-outer = orderings counted. **Pseudocode:**

```
0/1 KNAPSACK (each item once):
    dp[0] = 1
    for each item i:
        for j from target DOWN TO nums[i]:     // DESCENDING reads OLD dp (item i not yet used)
            dp[j] += dp[j - nums[i]]

UNBOUNDED KNAPSACK (item reusable):
    dp[0] = 1
    for each item i:
        for j from nums[i] UP TO target:       // ASCENDING reads NEW dp (item i already used this pass)
            dp[j] += dp[j - nums[i]]

PERMUTATION COUNT (order matters):
    dp[0] = 1
    for j from 1 to target:                     // OUTER = target value
        for each num in nums:                   // INNER = try every num, so orderings differ
            if j >= num: dp[j] += dp[j - num]
```

```java
// ── 0/1 KNAPSACK ── each item at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = target; j >= nums[i]; j--)   // DESCENDING: sees OLD dp (before item i)
        dp[j] += dp[j - nums[i]];
}

// ── UNBOUNDED KNAPSACK ── each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = nums[i]; j <= target; j++)    // ASCENDING: sees NEW dp (after item i already used)
        dp[j] += dp[j - nums[i]];
}

// ── PERMUTATION COUNT ── order matters (Combination Sum IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target; j++) {           // OUTER: target
    for (int num : nums) {                    // INNER: try all nums (not fixing order)
        if (j >= num) dp[j] += dp[j - num];
    }
}
```

Why descending for 0/1:
`dp[j - nums[i]]` is read **before** `dp[j]` is updated (j > j-nums[i]). So it always sees the dp state from before item `i` was considered — no item is used twice.

Why ascending for unbounded:
`dp[j - nums[i]]` was already updated in this same pass (j - nums[i] < j, processed earlier). This allows item `i` to be picked multiple times.

2D anchor: both loops are `dp[i][j] = dp[i-1][j] + dp[i-?][j-w]` collapsed to 1D — descending keeps `dp[j-w]` on the *previous* row (item i unused), ascending pulls it from the *current* row (item i reused).

Mnemonic: DESCENDING = **D**istinct (each item once), ASCENDING = **A**gain (reusable).

Flavor = seed + operator (same skeleton):

- count → `dp[0]=1`, `dp[j] += dp[j-w]`
- feasibility → `dp[0]=true`, `dp[j] = dp[j] || dp[j-w]`
- max-value → `dp[0]=0`, `dp[j] = Math.max(dp[j], dp[j-w] + v)`

#416 Partition Equal Subset Sum

Description: Can the array be partitioned into two subsets with equal sum?
Key: equivalent to: can we pick a subset summing to `total/2`?

Variables: `dp[j]` = can some subset of the items seen so far sum exactly to `j`. **Pseudocode:**

```
total = sum of nums; if odd, impossible -> return false
target = total / 2
dp[0..target] = false; dp[0] = true        // empty subset sums to 0
for each num:                               // consider item once (0/1)
    for j from target down to num:          // DESCENDING -> 0/1: dp[j-num] is from before this item
        dp[j] = dp[j] OR dp[j-num]          // keep j, or add num onto a subset summing to j-num
return dp[target]
```

```java
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = Arrays.stream(nums).sum();
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int j = target; j >= num; j--)   // 0/1: descending
                dp[j] = dp[j] || dp[j - num];
        }
        return dp[target];
    }
}
```

#494 Target Sum

Description: Assign `+` or `-` to each number to reach `target`. How many ways?
Key: assign `+` to subset P and `-` to subset N. Then `P - N = target` and `P + N = sum`. So `P = (sum + target) / 2`. Count subsets summing to P.

Variables: `dp[j]` = number of distinct subsets (of items seen so far) that sum exactly to `j`. **Pseudocode:**

```
sum = sum of nums
if (sum+target) is odd or |target| > sum: return 0
t = (sum + target) / 2                      // P = subset assigned '+'
dp[0..t] = 0; dp[0] = 1                      // one way to make 0: empty subset
for each num:                               // consider item once (0/1)
    for j from t down to num:               // DESCENDING -> 0/1: count each item at most once
        dp[j] += dp[j-num]                   // add the ways that made j-num, now including num
return dp[t]
```

```java
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = Arrays.stream(nums).sum();
        if ((sum + target) % 2 != 0 || Math.abs(target) > sum) return 0;
        int t = (sum + target) / 2;
        int[] dp = new int[t + 1];
        dp[0] = 1;
        for (int num : nums) {
            for (int j = t; j >= num; j--)        // 0/1: descending
                dp[j] += dp[j - num];
        }
        return dp[t];
    }
}
```

#474 Ones and Zeroes

Description: Given strings of '0'/'1', find the largest subset using at most `m` zeros and `n` ones.
Key: 2D 0/1 knapsack — two capacity dimensions. Both must iterate descending.

Variables: `dp[i][j]` = max size of a subset of strings seen so far that uses at most `i` zeros and `j` ones.
Pseudocode:

```
dp[0..m][0..n] = 0                          // empty subset has size 0
for each string s:                          // consider item once (0/1)
    zeros = count of '0' in s; ones = count of '1' in s
    for i from m down to zeros:             // DESCENDING both dims -> 0/1: each string used once
        for j from n down to ones:
            dp[i][j] = max(dp[i][j],         // skip s
                           dp[i-zeros][j-ones] + 1)   // take s, spend its zeros/ones
return dp[m][n]
```

```java
class Solution {
    public int findMaxForm(String[] strs, int m, int n) {
        int[][] dp = new int[m + 1][n + 1];
        for (String s : strs) {
            int zeros = 0, ones = 0;
            for (char c : s.toCharArray()) { if (c == '0') zeros++; else ones++; }
            for (int i = m; i >= zeros; i--)          // 0/1: descending (both dims)
                for (int j = n; j >= ones; j--)
                    dp[i][j] = Math.max(dp[i][j], dp[i - zeros][j - ones] + 1);
        }
        return dp[m][n];
    }
}
```

#322 Coin Change

Description: Minimum number of coins to make `amount`. Coins can be reused.
Key: unbounded knapsack (ascending), but minimize instead of count — seed with INF, take min.

Variables: `dp[j]` = minimum number of coins needed to make amount `j` (sentinel `amount+1` = impossible).
Pseudocode:

```
dp[0..amount] = amount+1 (impossible sentinel); dp[0] = 0   // 0 coins make amount 0
for each coin:                              // unbounded: coin reusable
    for j from coin up to amount:           // ASCENDING -> unbounded: dp[j-coin] may already include thi
        dp[j] = min(dp[j], dp[j-coin] + 1)  // reuse best way to make j-coin, add one more coin
return dp[amount] > amount ? -1 : dp[amount]
```

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);               // impossible sentinel
        dp[0] = 0;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++)   // unbounded: ascending
                dp[j] = Math.min(dp[j], dp[j - coin] + 1);
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

#518 Coin Change II

Description: Number of distinct combinations (order doesn't matter) to make `amount`.
Key: unbounded knapsack (ascending), count variant — combinations, not permutations.

Variables: `dp[j]` = number of distinct combinations (order ignored) of coins that sum to `j`. **Pseudocode:**

```
dp[0..amount] = 0; dp[0] = 1                // one way to make 0: pick nothing
for each coin:                              // coin OUTER fixes coin order -> combinations, no duplicates
    for j from coin up to amount:           // ASCENDING -> unbounded: coin reusable
        dp[j] += dp[j-coin]                  // add combinations of j-coin extended by this coin
return dp[amount]
```

```java
class Solution {
    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++)   // unbounded: ascending
                dp[j] += dp[j - coin];
        }
        return dp[amount];
    }
}
```

#377 Combination Sum IV

Description: Number of ordered sequences (order matters) that sum to `target`.
Key: permutation count — swap loop order: target outer, nums inner. This tries all nums for each partial sum, so different orderings are counted separately.

Variables: `dp[j]` = number of ordered sequences (order matters) of nums that sum to `j`. **Pseudocode:**

```
dp[0..target] = 0; dp[0] = 1                // one empty sequence sums to 0
for j from 1 up to target:                  // TARGET OUTER -> permutations: every num retried at each j
    for each num:                           // num INNER tries all numbers as the LAST element
        if j >= num: dp[j] += dp[j-num]      // sequences making j-num, each appending num
return dp[target]
```

```java
class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        dp[0] = 1;
        for (int j = 1; j <= target; j++) {        // outer: target value
            for (int num : nums) {                  // inner: all numbers
                if (j >= num) dp[j] += dp[j - num];
            }
        }
        return dp[target];
    }
}
```

518 vs 377 — Side by Side

```
                    #518 Coin Change II         #377 Combination Sum IV
Loop order:         coins outer, j inner        j outer, nums inner
Counts:             combinations (sets)         permutations (sequences)
Example (1,2→3):    {1,2} and {1,1,1} = 2      {1,2},{2,1},{1,1,1} = 3
Why:                fixing coin order = no dup  re-trying all nums each j = all orders
```

Part 3 — 2D Sequence DP

Canonical Template

Variables: `dp[i][j]` = the answer for the prefixes `s1[0..i)` (first `i` chars) and `s2[0..j)` (first `j` chars); index `0` = empty prefix. **Pseudocode:**

```
dp = (m+1) x (n+1) table
initialize dp[0][*] and dp[*][0] from the empty-prefix base cases
for i from 1 to m:                          // each char of s1
    for j from 1 to n:                      // each char of s2
        if s1[i-1] == s2[j-1]:              // current chars match
            dp[i][j] = dp[i-1][j-1] + 1     // extend the diagonal (both prefixes shrink by 1)
        else:
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])  // drop a char from one/both side
return dp[m][n]
```

```java
// i indexes string/array 1 (1-indexed), j indexes string/array 2
int[][] dp = new int[m + 1][n + 1];
// Init dp[0][*] and dp[*][0] based on problem
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) {
            dp[i][j] = dp[i-1][j-1] + 1;          // characters match
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }
}
```

#1143 Longest Common Subsequence

Description: Length of the longest subsequence present in both strings.
Intuition: if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.

Variables: `dp[i][j]` = LCS length of `text1[0..i)` and `text2[0..j)` (prefixes of lengths `i` and `j`); row/col 0 = empty prefix = 0. **Pseudocode:**

```
m = len(text1); n = len(text2)
dp = (m+1) x (n+1) table, all zero        // dp[0][*]=dp[*][0]=0 (empty prefix)
for i from 1 to m:
    for j from 1 to n:
        if text1[i-1] == text2[j-1]:       // last chars match
            dp[i][j] = dp[i-1][j-1] + 1    // extend LCS on the diagonal
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])  // drop one char, keep better
return dp[m][n]
```

```java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i-1) == text2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#72 Edit Distance

Description: Minimum operations (insert, delete, replace) to convert `word1` to `word2`.
Init: `dp[i][0] = i` (delete all), `dp[0][j] = j` (insert all).
Intuition: matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.

Variables: `dp[i][j]` = min edits to convert `word1[0..i)` to `word2[0..j)`; init `dp[i][0]=i` (delete all), `dp[0][j]=j` (insert all). **Pseudocode:**

```
m = len(word1); n = len(word2)
dp = (m+1) x (n+1) table
for i from 0 to m: dp[i][0] = i            // delete i chars
for j from 0 to n: dp[0][j] = j            // insert j chars
for i from 1 to m:
    for j from 1 to n:
        if word1[i-1] == word2[j-1]:        // match: no cost
            dp[i][j] = dp[i-1][j-1]
        else:
            dp[i][j] = 1 + min(dp[i-1][j-1],    // replace
                               dp[i-1][j],       // delete
                               dp[i][j-1])       // insert
return dp[m][n]
```

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1];                          // match: no cost
                } else {
                    dp[i][j] = 1 + Math.min(dp[i-1][j-1],             // replace
                                   Math.min(dp[i-1][j],               // delete
                                            dp[i][j-1]));             // insert
                }
            }
        }
        return dp[m][n];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#115 Distinct Subsequences

Description: Number of ways `s` contains `t` as a subsequence.
Init: `dp[i][0] = 1` (empty t matched by any s prefix).
Intuition: you can always skip the current char of s; if it matches the current char of t, you may also use it to advance t.

Variables: `dp[i][j]` = number of ways `s[0..i)` contains `t[0..j)` as a subsequence; init `dp[i][0]=1` (empty `t` matched by any `s` prefix). **Pseudocode:**

```
m = len(s); n = len(t)
dp = (m+1) x (n+1) table
for i from 0 to m: dp[i][0] = 1            // empty t always matched (1 way)
for i from 1 to m:
    for j from 1 to n:
        dp[i][j] = dp[i-1][j]              // always: skip s[i-1]
        if s[i-1] == t[j-1]:
            dp[i][j] += dp[i-1][j-1]       // also: use s[i-1] to match t[j-1]
return dp[m][n]
```

```java
class Solution {
    public int numDistinct(String s, String t) {
        int m = s.length(), n = t.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = 1;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = dp[i-1][j];                                // always: skip s[i]
                if (s.charAt(i-1) == t.charAt(j-1))
                    dp[i][j] += dp[i-1][j-1];                         // also: use s[i] to match t[j]
            }
        }
        return dp[m][n];
    }
}
```

Time O(m·n) | **Space** O(m·n)

1143 vs 72 vs 115 — Transition Cheat Sheet

```
                    #1143 LCS           #72 Edit Distance       #115 Distinct Subseq
Match (s[i]==t[j]): dp[i-1][j-1]+1      dp[i-1][j-1]            dp[i-1][j]+dp[i-1][j-1]
No match:           max(up, left)        1+min(diag,up,left)     dp[i-1][j]  (skip s[i])
Init dp[0][*]:      0                   j (insert j chars)       0
Init dp[*][0]:      0                   i (delete i chars)       1 (empty t always matches)
```

Part 4 — Interval DP

Canonical Template

Rule: `i` goes right-to-left (ensures shorter/inner intervals are computed first). `j` goes left-to-right from `i+1`. When computing `dp[i][j]`, all `dp[i'][j']` with `i' > i` or `j' < j` are already done.

Variables: `dp[i][j]` = the answer for the interval `[i, j]` (left endpoint `i`, right endpoint `j`). **Pseudocode:**

```
dp = n x n table
for each i: dp[i][i] = base_case            // length-1 intervals
for i from n-1 down to 0:                    // i RIGHT-TO-LEFT so dp[i+1][*] (inner) is ready
    for j from i+1 to n-1:                   // j LEFT-TO-RIGHT from i+1 so dp[i][j-1] is ready
        // safe to read dp[i+1][j], dp[i][j-1], dp[i+1][j-1] (all strictly smaller intervals)
        dp[i][j] = ...transition over interval [i,j]...
```

```java
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case;   // single elements
for (int i = n - 1; i >= 0; i--) {                   // right-to-left
    for (int j = i + 1; j < n; j++) {                // left-to-right from i+1
        // dp[i][j] can safely use dp[i+1][j], dp[i][j-1], dp[i+1][j-1]
        dp[i][j] = ...;
    }
}
```

Alternative (forward `i`, inner `j` descending): `i` is the right endpoint going left-to-right; `j` is the left endpoint sweeping back from `i-1`. Inner intervals (larger `j`, smaller `i`) are already filled.

Variables: `dp[i][j]` = the answer for the interval `[j, i]` (right endpoint `i`, left endpoint `j`). **Pseudocode:**

```
dp = n x n table
for each i: dp[i][i] = base_case            // length-1 intervals
for i from 0 to n-1:                         // i = RIGHT endpoint, FORWARD left-to-right
    for j from i-1 down to 0:                // j = LEFT endpoint, DESCENDING from i-1 to 0
        // safe to read dp[i-1][j], dp[i][j+1], dp[i-1][j+1] (all shorter intervals already filled)
        dp[i][j] = ...transition over interval [j,i]...
```

```java
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case;   // single elements
for (int i = 0; i < n; i++) {                        // i = right endpoint, left-to-right
    for (int j = i - 1; j >= 0; j--) {               // j = left endpoint, from i-1 down to 0
        // dp[i][j] can safely use dp[i-1][j], dp[i][j+1], dp[i-1][j+1]
        dp[i][j] = ...;
    }
}
```

Alternative (triangular fill, column base case): used when `dp[i][j]` depends only on the previous row/column (e.g. counting problems). Fill row by row with `j` from `0` to `i`.

Variables: `dp[i][j]` = the count/answer for the `(i, j)` subproblem, defined only for `j <= i` (lower-triangular).
Pseudocode:

```
dp = n x n table
for each i: dp[i][0] = 1                    // column base case; dp[0][j>0] = 0
for i from 1 to n-1:                        // fill ROW BY ROW, top to bottom
    for j from 1 to i:                      // j from 1 up to i (triangular, j <= i)
        // safe to read dp[i-1][j], dp[i][j-1], dp[i-1][j-1] (prior row / earlier in row)
        dp[i][j] = ...transition...
```

```java
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][0] = 1;   // base: dp[i][0] = 1; dp[0][j] = 0 for j > 0
for (int i = 1; i < n; i++) {
    for (int j = 1; j <= i; j++) {          // j from 1 to i
        // dp[i][j] can safely use dp[i-1][j], dp[i][j-1], dp[i-1][j-1]
        dp[i][j] = ...;
    }
}
```

#647 Palindromic Substrings

Description: Count all palindromic substrings of `s`.
Intuition: `s[i..j]` is a palindrome when its two ends match AND the inside `s[i+1..j-1]` already is.

Variables: `dp[i][j]` = whether the substring `s[i..j]` (inclusive) is a palindrome; `count` = running total of palindromic substrings. **Pseudocode:**

```
dp = n x n boolean table; count = 0
for i from n-1 down to 0:                   // i RIGHT-TO-LEFT so inner dp[i+1][*] is ready
    for j from i to n-1:                    // j from i (length >= 1) to n-1
        dp[i][j] = (s[i] == s[j])           // ends must match
                   AND (j-i < 2 OR dp[i+1][j-1])   // length <= 2, or inner already palindrome
        if dp[i][j]: count++                // every true cell is one palindromic substring
return count
```

```java
class Solution {
    public int countSubstrings(String s) {
        int n = s.length(), count = 0;
        boolean[][] dp = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i; j < n; j++) {
                dp[i][j] = s.charAt(i) == s.charAt(j)
                        && (j - i < 2 || dp[i+1][j-1]);   // ends match && inner is palindrome
                if (dp[i][j]) count++;
            }
        }
        return count;
    }
}
```

Time O(n²) | **Space** O(n²)

#516 Longest Palindromic Subsequence

Description: Length of the longest subsequence of `s` that is a palindrome.
Intuition: matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.

Variables: `dp[i][j]` = length of the longest palindromic subsequence in `s[i..j]` (inclusive); single chars `dp[i][i]=1`.
Pseudocode:

```
n = len(s)
dp = n x n table
for each i: dp[i][i] = 1                    // single char is a palindrome of length 1
for i from n-1 down to 0:                    // i RIGHT-TO-LEFT so inner dp[i+1][*] ready
    for j from i+1 to n-1:                   // j LEFT-TO-RIGHT from i+1
        if s[i] == s[j]:                     // matching ends
            dp[i][j] = dp[i+1][j-1] + 2      // add 2 to inner range's best
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1])  // drop one end, keep larger
return dp[0][n-1]
```

```java
class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) dp[i][i] = 1;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return dp[0][n-1];
    }
}
```

Time O(n²) | **Space** O(n²)

#312 Burst Balloons

Description: Burst all balloons to maximize coins. Coins from bursting balloon `k` between open boundaries `i` and `j` = `balloons[i] * balloons[k] * balloons[j]`.
Key insight: think of `k` as the **last** balloon burst in range `(i, j)` (open interval). Adding sentinel `1`s at both ends simplifies boundary cases.
Template: length-based outer loop; inner loop over last-burst position `k`.

Variables: `dp[i][j]` = max coins from bursting all balloons strictly inside the open interval `(i, j)`; `b` = nums padded with sentinel `1`s at both ends; `k` = the LAST balloon burst in `(i, j)`. **Pseudocode:**

```
n = len(nums)
b = array of size n+2; b[0] = b[n+1] = 1   // sentinel 1s at both ends
copy nums into b[1..n]
size = n+2
dp = size x size table, all zero
for len from 2 to size-1:                    // interval length (open), needs >=1 inside
    for i from 0 while i+len < size:          // i = left open boundary
        j = i + len                           // j = right open boundary
        for k from i+1 to j-1:                // k = LAST balloon burst in (i, j)
            dp[i][j] = max(dp[i][j],
                           dp[i][k] + b[i]*b[k]*b[j] + dp[k][j])
return dp[0][n+1]
```

```java
class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] b = new int[n + 2];
        b[0] = b[n + 1] = 1;
        for (int i = 0; i < n; i++) b[i + 1] = nums[i];
        int size = n + 2;
        int[][] dp = new int[size][size];
        for (int len = 2; len < size; len++) {
            for (int i = 0; i + len < size; i++) {
                int j = i + len;
                for (int k = i + 1; k < j; k++) {   // k = last balloon burst in (i, j)
                    dp[i][j] = Math.max(dp[i][j],
                        dp[i][k] + b[i] * b[k] * b[j] + dp[k][j]);
                }
            }
        }
        return dp[0][n + 1];
    }
}
```

Time $O(n^3)$ | **Space** $O(n^2)$

Part 5 — Grid DP

Canonical Template

Variables: `dp[i][j]` = best (path count / cost) to reach cell `(i, j)` moving only right/down; `memo[i][j]` = longest path starting from `(i, j)` for the all-direction DFS variant; `dr` / `dc` = 4-direction deltas. **Pseudocode:**

```
dr = {1,-1,0,0}; dc = {0,0,1,-1}            // DOWN UP RIGHT LEFT

// Standard grid (only right/down — DAG, no cycle):
dp = rows x cols table
initialize first row and first column
for i from 0 to rows-1:
    for j from 0 to cols-1:
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1])   // combine top & left

// All-direction grid (memoized DFS for increasing/decreasing paths):
memo = rows x cols table
for i from 0 to rows-1:
    for j from 0 to cols-1:
        dfs(grid, i, j, memo, dr, dc)        // each cell caches its longest path
```

```
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};   // DOWN UP RIGHT LEFT

// Standard grid (only move right/down — DAG, no cycle):
int[][] dp = new int[rows][cols];
// initialize first row and first column
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1]);

// All-direction grid (memoized DFS for increasing/decreasing paths):
int[][] memo = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dfs(grid, i, j, memo, dr, dc);
```

#62 Unique Paths

Description: Number of paths from top-left to bottom-right, moving only right or down.
Init: first row and column are all 1 (only one way to reach any edge cell).
Intuition: you reach a cell only from above or from the left, so its path count is the sum of those two.

Variables: `dp[i][j]` = number of paths to reach cell `(i, j)` moving only right/down; first row and first column all `1`.
Pseudocode:

```
dp = m x n table
for i from 0 to m-1: dp[i][0] = 1          // one way down the first column
for j from 0 to n-1: dp[0][j] = 1          // one way across the first row
for i from 1 to m-1:
    for j from 1 to n-1:
        dp[i][j] = dp[i-1][j] + dp[i][j-1] // paths from above + paths from left
return dp[m-1][n-1]
```

```java
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#64 Minimum Path Sum

Description: Minimum sum path from top-left to bottom-right, moving only right or down.
Init: first row/column are cumulative sums (no choice on edges).
Intuition: the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.

Variables: `dp[i][j]` = minimum path sum to reach cell `(i, j)` moving only right/down; first row/column are cumulative sums. **Pseudocode:**

```
m = rows; n = cols
dp = m x n table
dp[0][0] = grid[0][0]
for i from 1 to m-1: dp[i][0] = dp[i-1][0] + grid[i][0]   // first column cumulative
for j from 1 to n-1: dp[0][j] = dp[0][j-1] + grid[0][j]   // first row cumulative
for i from 1 to m-1:
    for j from 1 to n-1:
        dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])  // own value + cheaper of top/left
return dp[m-1][n-1]
```

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
        for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
        for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
        return dp[m-1][n-1];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#221 Maximal Square

Description: Find the largest all-1 square in a binary matrix. Return its area.
Key insight: `dp[i][j]` = side length of the largest all-1 square with its bottom-right corner at `(i,j)`. Limited by the minimum of up, left, and diagonal neighbors.

Variables: `dp[i][j]` = side length of the largest all-1 square whose bottom-right corner is `(i, j)`; `maxSide` = best side length seen. **Pseudocode:**

```
m = rows; n = cols; maxSide = 0
dp = m x n table
for i from 0 to m-1:
    for j from 0 to n-1:
        if matrix[i][j] == '1':
            if i == 0 or j == 0:
                dp[i][j] = 1                  // edge cell: square of side 1
            else:
                dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1  // limited by 3 neighbors
            maxSide = max(maxSide, dp[i][j])
return maxSide * maxSide                       // area
```

```java
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length, maxSide = 0;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == '1') {
                    dp[i][j] = (i == 0 || j == 0) ? 1
                             : Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1])) + 1;
                    maxSide = Math.max(maxSide, dp[i][j]);
                }
            }
        }
        return maxSide * maxSide;
    }
}
```

Time O(m·n) | **Space** O(m·n)

#329 Longest Increasing Path in a Matrix

Description: Find the longest strictly increasing path. Can move in all 4 directions.
Template: DFS + memoization (grid has ordering from values, so no cycle in the recursion DAG).
Intuition: strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.

Variables: `memo[i][j]` = length of the longest strictly increasing path STARTING at `(i, j)` (0 = uncomputed); `max` = global best; `dr` / `dc` = 4-direction deltas. **Pseudocode:**

```
m = rows; n = cols
memo = m x n table, all zero; max = 0
for each cell (i, j): max = max(max, dfs(i, j))   // try every start

dfs(i, j):
    if memo[i][j] != 0: return memo[i][j]          // cached
    memo[i][j] = 1                                  // at least the cell itself
    for each of 4 directions (ni, nj):
        if in bounds and matrix[ni][nj] > matrix[i][j]:   // strictly increasing
            memo[i][j] = max(memo[i][j], dfs(ni, nj) + 1)
    return memo[i][j]
```

```java
class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public int longestIncreasingPath(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        int[][] memo = new int[m][n];
        int max = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                max = Math.max(max, dfs(matrix, i, j, memo));
        return max;
    }
    private int dfs(int[][] matrix, int i, int j, int[][] memo) {
        if (memo[i][j] != 0) return memo[i][j];
        memo[i][j] = 1;
        for (int dir = 0; dir < 4; dir++) {
            int ni = i + dr[dir], nj = j + dc[dir];
            if (ni >= 0 && ni < matrix.length && nj >= 0 && nj < matrix[0].length
                    && matrix[ni][nj] > matrix[i][j])
                memo[i][j] = Math.max(memo[i][j], dfs(matrix, ni, nj, memo) + 1);
        }
        return memo[i][j];
    }
}
```

Time O(m·n) | **Space** O(m·n)

Part 6 — State Machine DP (Stock Problems)

Canonical States

```
cash      = max profit when NOT holding (free to buy or idle)
hold      = max profit when HOLDING stock (bought it at some cost)
cooldown  = max profit right after selling (can't buy today)
```

All Five Transitions

```
                   buy                  sell                re-buy condition
#121 (1 tx):       hold = max(hold, -price)                 no re-buy (ignore cash)
#122 (unlimited):  hold = max(hold, cash - price)           re-buy with full cash
#123 (2 tx):       chain: buy2 uses sell1 cash              4 states chained
#309 (cooldown):   hold = max(hold, cooldown - price)       must wait 1 day
#714 (fee):        hold = max(hold, cash - price)           pay fee on sell
```

#121 Best Time to Buy and Sell Stock (at most 1 transaction)

Intuition: `hold` is the best profit if you currently own a share bought at the lowest price seen; sell whenever today's price beats that.

Variables: `cash` = max profit while NOT holding; `hold` = max profit while HOLDING (one buy, no reinvestment of prior profit). **Pseudocode:**

```
cash = 0; hold = -prices[0]                 // day 0: bought at prices[0]
for i from 1 to n-1:
    cash = max(cash, hold + prices[i])      // sell today or stay out
    hold = max(hold, -prices[i])            // buy today (NO cash added: only 1 buy)
return cash
```

```java
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, -prices[i]);         // no cash reinvestment (1 buy)
        }
        return cash;
    }
}
```

Time O(n) | **Space** O(1)

#122 Best Time to Buy and Sell Stock II (unlimited transactions)

Key difference from #121: `hold = max(hold, cash - prices[i])` — can reinvest all prior profits.
Intuition: with unlimited trades, buying can fund itself from accumulated `cash`, so capture every upward move.

Variables: `cash` = max profit while NOT holding; `hold` = max profit while HOLDING, where buying reinvests accumulated `cash` (unlimited transactions). **Pseudocode:**

```
cash = 0; hold = -prices[0]
for i from 1 to n-1:
    cash = max(cash, hold + prices[i])      // sell today or stay out
    hold = max(hold, cash - prices[i])      // buy today funded by prior cash (reinvest)
return cash
```

```java
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, cash - prices[i]);   // reinvest cash (unlimited buys)
        }
        return cash;
    }
}
```

Time O(n) | **Space** O(1)

#123 Best Time to Buy and Sell Stock III (at most 2 transactions)

Key: 4 named states chained in order: buy1 → sell1 → buy2 → sell2.
Intuition: the second buy can only spend profit left after the first sell, so chain the states in trade order.

Variables: `buy1` / `sell1` / `buy2` / `sell2` = best profit after the 1st buy / 1st sell / 2nd buy / 2nd sell, chained in trade order (at most 2 transactions). **Pseudocode:**

```
buy1 = -prices[0]; sell1 = 0; buy2 = -prices[0]; sell2 = 0
for i from 1 to n-1:
    buy1  = max(buy1,  -prices[i])          // 1st buy
    sell1 = max(sell1, buy1  + prices[i])   // 1st sell (uses buy1)
    buy2  = max(buy2,  sell1 - prices[i])   // 2nd buy (spends sell1 profit)
    sell2 = max(sell2, buy2  + prices[i])   // 2nd sell (uses buy2)
return sell2
```

```java
class Solution {
    public int maxProfit(int[] prices) {
        int buy1 = -prices[0], sell1 = 0, buy2 = -prices[0], sell2 = 0;
        for (int i = 1; i < prices.length; i++) {
            buy1  = Math.max(buy1,  -prices[i]);
            sell1 = Math.max(sell1,  buy1  + prices[i]);
            buy2  = Math.max(buy2,   sell1 - prices[i]);
            sell2 = Math.max(sell2,  buy2  + prices[i]);
        }
        return sell2;
    }
}
```

Time O(n) | **Space** O(1)

#309 Best Time to Buy and Sell Stock with Cooldown

Description: After selling, must wait 1 day before buying again.
Key: `hold` can only use `cooldown` (yesterday's `cash`), not today's `cash`. Save `prevCash` before updating.
Intuition: the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.

Variables: `cash` = max profit not holding; `hold` = max profit holding; `cooldown` = yesterday's `cash` (the only cash a buy may draw from); `prevCash` = today's `cash` saved before it is overwritten. **Pseudocode:**

```
cash = 0; hold = -prices[0]; cooldown = 0
for i from 1 to n-1:
    prevCash = cash                          // save today's cash before update
    cash     = max(cash, hold + prices[i])   // sell today or stay out
    hold     = max(hold, cooldown - prices[i])  // buy only from cooldown (>=1 day old cash)
    cooldown = prevCash                       // tomorrow's cooldown = yesterday's cash
return cash
```

```java
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0], cooldown = 0;
        for (int i = 1; i < prices.length; i++) {
            int prevCash = cash;
            cash     = Math.max(cash,     hold + prices[i]);
            hold     = Math.max(hold,     cooldown - prices[i]);  // buy only from cooldown state
            cooldown = prevCash;                                  // yesterday's cash
        }
        return cash;
    }
}
```

Time O(n) | **Space** O(1)

#714 Best Time to Buy and Sell Stock with Transaction Fee

Description: Pay a fee per transaction (on sell). Unlimited transactions.
Key: same as #122 but subtract `fee` when selling.
Intuition: the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.

Variables: `cash` = max profit not holding; `hold` = max profit holding (reinvests `cash` like #122); selling pays a flat `fee`.
Pseudocode:

```
cash = 0; hold = -prices[0]
for i from 1 to n-1:
    cash = max(cash, hold + prices[i] - fee) // sell today, pay fee
    hold = max(hold, cash - prices[i])       // buy today funded by prior cash (reinvest)
return cash
```

```java
class Solution {
    public int maxProfit(int[] prices, int fee) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i] - fee);   // pay fee on sell
            hold = Math.max(hold, cash - prices[i]);
        }
        return cash;
    }
}
```

Time O(n) | **Space** O(1)

Stock Problems — Differences at a Glance

```
Problem     hold update                  cash update                Extra state
#121        max(hold, -price)            max(cash, hold+price)      none
#122        max(hold, cash-price)        max(cash, hold+price)      none
#123        4 variables, chained          —                          sell1 feeds buy2
#309        max(hold, cooldown-price)    max(cash, hold+price)      cooldown = prevCash
#714        max(hold, cash-price)        max(cash, hold+price-fee)  none
```

Template Chooser

Signal in problem	Template
`dp[i]` depends on `dp[i-1]`, `dp[i-2]`	Linear 1D — fixed lookback
`dp[i]` depends on all `dp[j]` where `j < i`	Look-back 1D — O(n²) inner loop
Each item used at most once, sum target	Knapsack 0/1 — j descending
Items reusable, sum target, order irrelevant	Knapsack Unbounded — j ascending
Items reusable, order matters (permutations)	Permutation count — target outer
Two strings/arrays, compare characters	2D Sequence DP
Palindrome, substring merge, balloon burst	Interval DP — i right-to-left
Grid, only right/down movement	Grid DP — cumulative from edges
Grid, all 4 directions, monotone constraint	DFS + memo (no cycle in DAG)
Buy/sell/hold decision changes day to day	State Machine — named states

#746 Min Cost Climbing Stairs

Description: Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).

Algorithm: 1D DP. `dp[i]` = min cost to reach step `i`. `dp[0] = dp[1] = 0` (can start for free). For `i >= 2`: `dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])`.

Variables: `dp[i]` = min cost to reach step `i` (with the top being step `n`); `cost[i]` = cost paid to leave step `i`; `n` = number of steps. **Pseudocode:**

```
n = number of steps
make dp of size n+1, dp[0] = dp[1] = 0   (free to start at step 0 or 1)
for i from 2 to n:
    dp[i] = min(reach i-1 then pay cost[i-1], reach i-2 then pay cost[i-2])
return dp[n]   (cost to reach the top)
```

```java
class Solution {
    public int minCostClimbingStairs(int[] cost) {
        int n = cost.length;
        int[] dp = new int[n + 1];
        for (int i = 2; i <= n; i++)
            dp[i] = Math.min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2]);  // ← two sources
        return dp[n];
    }
}
```

Time O(n) | **Space** O(n)

#91 Decode Ways

Description: A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.

Variation: Two sources at each position. If the current digit is non-zero, it can be decoded alone (`dp[i] += dp[i-1]`). If the previous two digits form a valid two-letter code (10-26), add `dp[i-2]` .

Variables: `dp[i]` = number of ways to decode the first `i` digits `s[0..i)` ; `oneDigit` = value of the single digit `s[i-1]` ; `twoDigit` = value of the two digits `s[i-2..i)` ; `n` = length of `s` . **Pseudocode:**

```
if first char is '0': return 0   (no valid decoding)
n = length of s
make dp of size n+1, dp[0] = 1, dp[1] = 1   (empty and first digit)
for i from 2 to n:
    oneDigit = value of s[i-1]
    twoDigit = value of s[i-2..i)
    if oneDigit != 0: dp[i] += dp[i-1]            (decode last digit alone)
    if 10 <= twoDigit <= 26: dp[i] += dp[i-2]     (decode last two digits together)
return dp[n]
```

```java
class Solution {
    public int numDecodings(String s) {
        if (s.charAt(0) == '0') return 0;
        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1; dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            int oneDigit = s.charAt(i-1) - '0';
            int twoDigit = Integer.parseInt(s.substring(i-2, i));
            if (oneDigit != 0) dp[i] += dp[i-1];               // ← VARIATION: single digit (1-9)
            if (twoDigit >= 10 && twoDigit <= 26) dp[i] += dp[i-2]; // ← VARIATION: two digits (10-26)
        }
        return dp[n];
    }
}
```

Time O(n) | **Space** O(n)

#279 Perfect Squares

Description: Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to `n` .

Algorithm: Unbounded knapsack. `dp[i]` = min squares to reach sum `i`. For each `i`, try all squares `j*j <= i`: `dp[i] = min(dp[i], dp[i - j*j] + 1)`.

Variables: `dp[i]` = minimum number of perfect squares summing to `i`; `j*j` = the perfect square being subtracted; `n` = target. **Pseudocode:**

```
make dp of size n+1, fill with infinity (n+1)
dp[0] = 0   (zero squares sum to 0)
for i from 1 to n:
    for each square j*j <= i:
        dp[i] = min(dp[i], dp[i - j*j] + 1)   (use one square j*j, plus best for remainder)
return dp[n]
```

```java
class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n + 1);
        dp[0] = 0;
        for (int i = 1; i <= n; i++)
            for (int j = 1; j * j <= i; j++)                    // ← VARIATION: iterate all squares ≤ i
                dp[i] = Math.min(dp[i], dp[i - j*j] + 1);
        return dp[n];
    }
}
```

Time O(n√n) | **Space** O(n)

#96 Unique Binary Search Trees

Description: Return the number of structurally unique BSTs that store values 1 to n.

Variation: Catalan number recurrence. `dp[i]` = number of unique BSTs with i nodes. When root = j, left subtree has j-1 nodes, right subtree has i-j nodes: `dp[i] += dp[j-1] * dp[i-j]`.

Variables: `dp[i]` = number of structurally unique BSTs holding `i` nodes (i-th Catalan number); `j` = which value is the root; `n` = number of nodes. **Pseudocode:**

```
make dp of size n+1, dp[0] = dp[1] = 1   (empty tree and single node)
for i from 2 to n:
    for each root choice j from 1 to i:
        dp[i] += dp[j-1] * dp[i-j]   (left subtree has j-1 nodes, right has i-j nodes)
return dp[n]
```

```java
class Solution {
    public int numTrees(int n) {
        int[] dp = new int[n + 1];
        dp[0] = dp[1] = 1;
        for (int i = 2; i <= n; i++)
            for (int j = 1; j <= i; j++)
                dp[i] += dp[j-1] * dp[i-j];    // ← VARIATION: Catalan: root=j, left=j-1, right=i-j
        return dp[n];
    }
}
```

Time O(n²) | **Space** O(n)

#63 Unique Paths II

Description: A robot moves from top-left to bottom-right in an m×n grid with obstacles. Count unique paths. Cells with `obstacleGrid[i][j] == 1` are blocked.

Variation: Same as #62 (Unique Paths) but skip cells with obstacles: `dp[i][j] = 0` if blocked, else `dp[i-1][j] + dp[i][j-1]`.

Variables: `dp[i][j]` = number of paths from the start to cell `(i,j)` avoiding obstacles; `obstacleGrid[i][j] == 1` marks a blocked cell; `m`, `n` = grid dimensions. **Pseudocode:**

```
m, n = grid dimensions
make dp of size m x n
fill first column with 1 until the first obstacle (then stays 0)
fill first row with 1 until the first obstacle (then stays 0)
for i from 1 to m-1:
    for j from 1 to n-1:
        if cell (i,j) is not an obstacle:
            dp[i][j] = dp[i-1][j] + dp[i][j-1]   (paths from above + from left)
return dp[m-1][n-1]
```

```java
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length, n = obstacleGrid[0].length;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1;  // ← VARIATION: stop at o
        for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                if (obstacleGrid[i][j] == 0)                    // ← VARIATION: skip blocked cells
                    dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#583 Delete Operation for Two Strings

Description: Return the minimum number of deletions to make `word1` and `word2` equal.

Variation: Reduce to LCS. Minimum deletions = `len(word1) + len(word2) - 2 * LCS(word1, word2)`. Compute LCS using standard 2D DP.

Variables: `dp[i][j]` = length of the longest common subsequence of `word1[0..i)` and `word2[0..j)`; `m`, `n` = lengths of the two words; answer = `m + n - 2*dp[m][n]`. **Pseudocode:**

```
m, n = lengths of word1, word2
make dp of size (m+1) x (n+1), all zero   (dp[i][j] = LCS length)
for i from 1 to m:
    for j from 1 to n:
        if word1[i-1] == word2[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1     (extend the common subsequence)
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])   (drop one char from either word)
return m + n - 2 * dp[m][n]   (delete everything outside the LCS)
```

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m+1][n+1];                        // dp[i][j] = LCS length
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return m + n - 2 * dp[m][n];                           // ← VARIATION: reduce to LCS
    }
}
```

Time O(m·n) | **Space** O(m·n)

#368 Largest Divisible Subset

Description: Find the largest subset of distinct positive integers such that for every pair (a, b), either `a % b == 0` or `b % a == 0`.

Variation: Sort first, then apply LIS-style DP. `dp[i]` = size of largest divisible subset ending at `nums[i]`. If `nums[i] % nums[j] == 0`, then `dp[i] = max(dp[i], dp[j] + 1)`. Track parent indices to reconstruct the subset.

Variables: `dp[i]` = size of the largest divisible subset ending at sorted `nums[i]`; `parent[i]` = previous element's index in that subset; `maxLen` / `maxIdx` = best subset length and its end index; `n` = count. **Pseudocode:**

```
sort nums ascending   (so divisibility forms a chain)
n = length, dp[i] = 1 for all, parent[i] = i for all
maxLen = 1, maxIdx = 0
for i from 1 to n-1:
    for j from 0 to i-1:
        if nums[i] divisible by nums[j] and dp[j]+1 > dp[i]:
            dp[i] = dp[j] + 1            (extend chain ending at j)
            parent[i] = j                (remember predecessor)
    if dp[i] > maxLen: maxLen = dp[i], maxIdx = i
walk back from maxIdx via parent, collecting nums until parent points to itself
reverse the collected list
return it
```

```java
class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums);                                          // ← VARIATION: sort required for divis
        int n = nums.length;
        int[] dp = new int[n], parent = new int[n];
        Arrays.fill(dp, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
        int maxLen = 1, maxIdx = 0;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) {  // ← VARIATION: divisibility check
                    dp[i] = dp[j] + 1;
                    parent[i] = j;
                }
            }
            if (dp[i] > maxLen) { maxLen = dp[i]; maxIdx = i; }
        }
        List<Integer> result = new ArrayList<>();
        int i = maxIdx;
        while (parent[i] != i) { result.add(nums[i]); i = parent[i]; }
        result.add(nums[i]);
        Collections.reverse(result);
        return result;
    }
}
```

Time O(n²) | **Space** O(n)

#931 Minimum Falling Path Sum

Description: Given an n×n matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.

Variation: Grid DP with three sources from the row above. `dp[i][j] = matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])`.

Variables: `dp[i][j]` = minimum falling-path sum that ends at cell `(i,j)`; `best` = cheapest reachable cell in the row above; `n` = matrix size. **Pseudocode:**

```
n = matrix size
make dp of size n x n
copy first row of matrix into dp[0]   (paths start anywhere in row 0)
for i from 1 to n-1:
    for j from 0 to n-1:
        best = dp[i-1][j]                           (straight down)
        if j > 0:   best = min(best, dp[i-1][j-1])  (diagonal up-left)
        if j < n-1: best = min(best, dp[i-1][j+1])  (diagonal up-right)
        dp[i][j] = matrix[i][j] + best
return the minimum value in the last row of dp
```

```java
class Solution {
    public int minFallingPathSum(int[][] matrix) {
        int n = matrix.length;
        int[][] dp = new int[n][n];
        dp[0] = matrix[0].clone();
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int best = dp[i-1][j];
                if (j > 0)   best = Math.min(best, dp[i-1][j-1]);  // ← VARIATION: three sources
                if (j < n-1) best = Math.min(best, dp[i-1][j+1]);
                dp[i][j] = matrix[i][j] + best;
            }
        }
        int min = Integer.MAX_VALUE;
        for (int val : dp[n-1]) min = Math.min(min, val);
        return min;
    }
}
```

Time O(n²) | **Space** O(n²)

#10 Regular Expression Matching

Description: Implement regex matching with `.` (any single char) and `*` (zero or more of the preceding element). Match must cover the entire input string. **Variation:** 2D DP. `dp[i][j]` = whether `s[0..i)` matches `p[0..j)`. The `*` has two cases: zero occurrences (`dp[i][j-2]`) or one-more occurrence when the preceding pattern char matches `s[i-1]`. **Variables:** `dp[i][j]` = true if `s[0..i)` matches pattern `p[0..j)`; `matches(i,j)` = does pattern char `p[j-1]` match string char `s[i-1]` (via `.` or equality); `m`, `n` = lengths of `s`, `p`. **Pseudocode:**

```
m, n = lengths of s, p
make dp of size (m+1) x (n+1), all false; dp[0][0] = true   (empty matches empty)
for j from 1 to n:
    if p[j-1] is '*': dp[0][j] = dp[0][j-2]   (x* erases itself against empty string)
for i from 1 to m:
    for j from 1 to n:
        if p[j-1] is '*':
            dp[i][j] = dp[i][j-2]                          (zero copies of preceding element)
            if preceding pattern char matches s[i-1]:
                dp[i][j] = dp[i][j] OR dp[i-1][j]          (consume one more s char, keep *)
        else if p[j-1] matches s[i-1]:
            dp[i][j] = dp[i-1][j-1]                        (single char/. match)
return dp[m][n]
matches(i,j): p[j-1] == '.' or s[i-1] == p[j-1]
```

```java
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int j = 1; j <= n; j++)
            if (p.charAt(j-1) == '*') dp[0][j] = dp[0][j-2];   // ← VARIATION: '*' can erase preceding
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p.charAt(j-1) == '*') {
                    dp[i][j] = dp[i][j-2];                      // ← VARIATION: zero occurrence
                    if (matches(s, p, i, j-1))
                        dp[i][j] = dp[i][j] || dp[i-1][j];      // ← VARIATION: one more occurrence
                } else if (matches(s, p, i, j)) {
                    dp[i][j] = dp[i-1][j-1];
                }
            }
        }
        return dp[m][n];
    }
    private boolean matches(String s, String p, int i, int j) {
        return p.charAt(j-1) == '.' || s.charAt(i-1) == p.charAt(j-1);
    }
}
```

Time O(m·n) | **Space** O(m·n)

#87 Scramble String

Description: Given s1 and s2, return true if s2 is a scrambled string of s1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them). **Variation:** memoized recursion over (i1, i2, len). At each length, try every split point both swapped and non-swapped. **Variables:** `memo[key]` = cached scramble result keyed by the pair `s1 + "#" + s2`; `count[26]` = character-frequency difference used to prune; `i` = split point; `n` = length of `s1`.
Pseudocode:

```
isScramble(s1, s2):
    if s1 == s2: return true
    if lengths differ: return false
    key = s1 + "#" + s2; if memoized, return cached value
    n = length of s1
    tally char counts of s1 minus s2; if any count nonzero: memo false, return false (different multiset)
    for split point i from 1 to n-1:
        no swap: left s1[0..i) vs s2[0..i) AND right s1[i..) vs s2[i..)
            if both scramble: memo true, return true
        swap: left s1[0..i) vs s2[n-i..) AND right s1[i..) vs s2[0..n-i)
            if both scramble: memo true, return true   (children were swapped)
    memo false, return false
```

```java
class Solution {
    private Map<String, Boolean> memo = new HashMap<>();
    public boolean isScramble(String s1, String s2) {
        if (s1.equals(s2)) return true;
        if (s1.length() != s2.length()) return false;
        String key = s1 + "#" + s2;
        if (memo.containsKey(key)) return memo.get(key);
        int n = s1.length();
        int[] count = new int[26];
        for (int i = 0; i < n; i++) { count[s1.charAt(i)-'a']++; count[s2.charAt(i)-'a']--; }
        for (int c : count) if (c != 0) { memo.put(key, false); return false; }  // char mismatch prune
        for (int i = 1; i < n; i++) {
            // no swap
            if (isScramble(s1.substring(0,i), s2.substring(0,i))
             && isScramble(s1.substring(i), s2.substring(i))) { memo.put(key, true); return true; }
            // swap                                              // ← VARIATION: try swapped partitions
            if (isScramble(s1.substring(0,i), s2.substring(n-i))
             && isScramble(s1.substring(i), s2.substring(0,n-i))) { memo.put(key, true); return true; }
        }
        memo.put(key, false); return false;
    }
}
```

Time $O(n^4)$ | **Space** $O(n^3)$

#1049 Last Stone Weight II

Description: Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone. **Variation:** equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum ≤ total/2. **Variables:** `dp[j]` = true if some subset of stones sums exactly to `j` (0/1 knapsack reachability); `total` = sum of all stones; `target` = total/2; answer minimizes `|S1 − S2| = total − 2j`.
Pseudocode:

```
total = sum of all stones
target = total / 2                      (largest group sum worth aiming for)
make dp of size target+1, all false; dp[0] = true
for each stone s:
    for j from target down to s:        (descending = use each stone once)
        dp[j] = dp[j] OR dp[j − s]      (reach j by adding stone s)
for j from target down to 0:
    if dp[j] is reachable: return total − 2*j   (minimal difference between groups)
return 0
```

```java
class Solution {
    public int lastStoneWeightII(int[] stones) {
        int total = 0;
        for (int s : stones) total += s;
        int target = total / 2;                    // ← VARIATION: subset sum target
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int s : stones)
            for (int j = target; j >= s; j--)       // ← 0/1 knapsack: j descending
                dp[j] = dp[j] || dp[j - s];
        for (int j = target; j >= 0; j--)
            if (dp[j]) return total - 2 * j;        // ← VARIATION: minimize |S1-S2|
        return 0;
    }
}
```

Time O(n·sum) | **Space** O(sum)

#1312 Minimum Insertion Steps to Make a String Palindrome

Description: Return the minimum number of insertions needed to make a string a palindrome. **Variation:** answer = n – longest palindromic subsequence (LPS). LPS = LCS of the string and its reverse; or solve directly via interval DP. **Variables:** `dp[i][j]` = length of the longest palindromic subsequence within `s[i..j]`; `n` = length of `s`; answer = `n - dp[0][n-1]` (chars not in the LPS must be matched by insertions). **Pseudocode:**

```
n = length of s
make dp of size n x n   (dp[i][j] = LPS length over s[i..j])
for i from n-1 down to 0:          (shorter spans computed first)
    dp[i][i] = 1                   (single char is a palindrome)
    for j from i+1 to n-1:
        if s[i] == s[j]:
            dp[i][j] = dp[i+1][j-1] + 2   (wrap matching ends around inner LPS)
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1])   (drop one end)
return n - dp[0][n-1]             (insertions = chars outside the LPS)
```

```java
class Solution {
    public int minInsertions(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];                 // dp[i][j] = LPS length in s[i..j]
        for (int i = n - 1; i >= 0; i--) {          // ← interval DP: shorter spans first
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return n - dp[0][n-1];                      // ← VARIATION: insertions = n - LPS
    }
}
```

Time O(n²) | **Space** O(n²)

Part 7 — Additional DP Problems

#44 Wildcard Matching

Description: Match string `s` against pattern `p` with `?` (any single char) and `*` (any sequence including empty). **Intuition:** `*` either consumes one more char of `s` (`dp[i-1][j]`) or matches empty (`dp[i][j-1]`); everything else is a 1-to-1 char/ `?` match. **Algorithm:** `dp[i][j]` = `s[0..i)` matches `p[0..j)` ; init `dp[0][j]` true while pattern is all `*` ; answer `dp[m][n]` .

Variables: `dp[i][j]` = true if `s[0..i)` matches pattern `p[0..j)` (with `?` = any single char, `*` = any sequence); `m` , `n` = lengths of `s` , `p` . **Pseudocode:**

```
m, n = lengths of s, p
make dp of size (m+1) x (n+1), all false; dp[0][0] = true   (empty matches empty)
for j from 1 to n:
    if p[j-1] is '*': dp[0][j] = dp[0][j-1]   (leading '*'s match the empty string)
for i from 1 to m:
    for j from 1 to n:
        if p[j-1] is '*':
            dp[i][j] = dp[i-1][j] OR dp[i][j-1]   (* consumes one s char, or matches empty)
        else if p[j-1] is '?' or p[j-1] == s[i-1]:
            dp[i][j] = dp[i-1][j-1]               (single-char match)
return dp[m][n]
```

```java
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int j = 1; j <= n; j++) {
            if (p.charAt(j-1) == '*') {
                dp[0][j] = dp[0][j-1];                          // ← VARIATION: '*' matches empty prefi
            }
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p.charAt(j-1) == '*') {
                    dp[i][j] = dp[i-1][j] || dp[i][j-1];        // ← VARIATION: '*' consumes char or ma
                } else if (p.charAt(j-1) == '?' || s.charAt(i-1) == p.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1];
                }
            }
        }
        return dp[m][n];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#53 Maximum Subarray

Description: Find the contiguous subarray with the largest sum. **Intuition:** the best sum ending at `i` either extends the previous best or restarts at `nums[i]` (Kadane). **Algorithm:** `current` = max sum ending at `i` = `max(nums[i], current + nums[i])` ; track global `result` .

Variables: `current` = maximum subarray sum ending exactly at index `i` (the collapsed `dp[i]`); `result` = best subarray sum seen overall. **Pseudocode:**

```
current = nums[0], result = nums[0]
for i from 1 to n-1:
    current = max(nums[i], current + nums[i])   (start fresh at i vs extend previous)
    result = max(result, current)               (track global best)
return result
```

```java
class Solution {
    public int maxSubArray(int[] nums) {
        int current = nums[0], result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            current = Math.max(nums[i], current + nums[i]);     // ← VARIATION: Kadane restart vs exten
            result = Math.max(result, current);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

Kadane's algorithm — the pattern behind #53. Keep `current` = best subarray sum **ending at i**; each step decide *start fresh* vs *extend*: `current = max(nums[i], current + nums[i])` (drop a negative prefix — it can only hurt), then `result = max(result, current)`. It's Linear-1D DP (`dp[i] = max(nums[i], dp[i-1] + nums[i])`) collapsed to O(1). Cousins: **#152** (track max **and** min — a negative flips them), **#918** circular (normal Kadane, or total − min-subarray for the wrap case).

#97 Interleaving String

Description: Return whether `s3` is formed by interleaving `s1` and `s2` while preserving each one's order. **Intuition:** the last char of `s3[0..i+j)` comes from either `s1` or `s2`; reuse the answer for the smaller prefix. **Algorithm:** `dp[i][j]` = `s1[0..i)` + `s2[0..j)` interleave to `s3[0..i+j)`; answer `dp[m][n]`.

Variables: `dp[i][j]` = true if `s1[0..i)` and `s2[0..j)` interleave to form `s3[0..i+j)`; `m`, `n` = lengths of `s1`, `s2`. **Pseudocode:**

```
m, n = lengths of s1, s2
if m + n != length of s3: return false   (lengths must add up)
make dp of size (m+1) x (n+1), all false; dp[0][0] = true
fill first column: dp[i][0] true while s1's prefix equals s3's prefix
fill first row:    dp[0][j] true while s2's prefix equals s3's prefix
for i from 1 to m:
    for j from 1 to n:
        dp[i][j] = (dp[i-1][j] and s1[i-1] == s3[i+j-1])    (take next char from s1)
                or (dp[i][j-1] and s2[j-1] == s3[i+j-1])    (take next char from s2)
return dp[m][n]
```

```java
class Solution {
    public boolean isInterleave(String s1, String s2, String s3) {
        int m = s1.length(), n = s2.length();
        if (m + n != s3.length()) {
            return false;
        }
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int i = 1; i <= m; i++) {
            dp[i][0] = dp[i-1][0] && s1.charAt(i-1) == s3.charAt(i-1);
        }
        for (int j = 1; j <= n; j++) {
            dp[0][j] = dp[0][j-1] && s2.charAt(j-1) == s3.charAt(j-1);
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = (dp[i-1][j] && s1.charAt(i-1) == s3.charAt(i+j-1))   // ← VARIATION: take fr
                        || (dp[i][j-1] && s2.charAt(j-1) == s3.charAt(i+j-1));  // ← VARIATION: take fr
            }
        }
        return dp[m][n];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#120 Triangle

Description: Find the minimum path sum from top to bottom, moving to adjacent indices on the row below. **Intuition:** working bottom-up, each cell's best is its value plus the cheaper of the two children directly below. **Algorithm:** `dp[j]` = min path sum from row `i` cell `j` to the bottom; collapse one row at a time; answer `dp[0]`.

Variables: `dp[j]` = minimum path sum from the current cell in column `j` down to the bottom (rolling 1D array reused per row); `n` = number of rows. **Pseudocode:**

```
n = number of rows
make dp of size n+1, all zero   (one extra slot so dp[j+1] is valid)
for i from n-1 down to 0:        (process rows bottom-up)
    for j from 0 to i:
        dp[j] = triangle[i][j] + min(dp[j], dp[j+1])   (add cheaper of the two cells below)
return dp[0]   (best total from the apex)
```

```java
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) {                    // ← VARIATION: bottom-up over rows
            for (int j = 0; j <= i; j++) {
                dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j+1]);
            }
        }
        return dp[0];
    }
}
```

Time O(n²) | **Space** O(n)

#132 Palindrome Partitioning II

Description: Return the minimum number of cuts so that every substring of the partition is a palindrome. **Intuition:** if `s[j..i]` is a palindrome, a cut after `j-1` lets `cuts[i] = cuts[j-1] + 1`; precompute palindromes with interval DP. **Algorithm:** `pal[i][j]` palindrome flag; `cuts[i]` = min cuts for `s[0..i]`; answer `cuts[n-1]`.

Variables: `pal[i][j]` = true if `s[i..j]` is a palindrome; `cuts[i]` = minimum cuts needed so every piece of `s[0..i]` is a palindrome; `n` = length of `s`. **Pseudocode:**

```
n = length of s
make pal of size n x n
for i from n-1 down to 0:
    for j from i to n-1:
        pal[i][j] = (s[i] == s[j]) and (span < 2 or inner pal[i+1][j-1])   (precompute palindromes)
make cuts of size n
for i from 0 to n-1:
    if s[0..i] is itself a palindrome: cuts[i] = 0
    else:
        cuts[i] = i                              (worst case: cut between every char)
        for j from 1 to i:
            if s[j..i] is a palindrome:
                cuts[i] = min(cuts[i], cuts[j-1] + 1)   (one cut before this palindromic suffix)
return cuts[n-1]
```

```java
class Solution {
    public int minCut(String s) {
        int n = s.length();
        boolean[][] pal = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--) {                    // ← VARIATION: interval DP precompute
            for (int j = i; j < n; j++) {
                pal[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 2 || pal[i+1][j-1]);
            }
        }
        int[] cuts = new int[n];
        for (int i = 0; i < n; i++) {
            if (pal[0][i]) {
                cuts[i] = 0;
            } else {
                cuts[i] = i;
                for (int j = 1; j <= i; j++) {
                    if (pal[j][i]) {
                        cuts[i] = Math.min(cuts[i], cuts[j-1] + 1);  // ← VARIATION: cut before palindr
                    }
                }
            }
        }
        return cuts[n-1];
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#140 Word Break II

Description: Return all sentences obtained by segmenting `s` into space-separated dictionary words. **Intuition:** memoize on each suffix the list of valid segmentations, prepending each matching word to the segmentations of the

remainder. **Algorithm:** `dfs(start)` returns all sentences for `s[start..]`; cache per `start`; answer `dfs(0)`.

Variables: `memo[start]` = cached list of all sentences segmenting the suffix `s[start..]`; `words` = dictionary set; `word` = candidate prefix; `tail` = a segmentation of the remainder. **Pseudocode:**

```
words = set of dictionary words
return dfs(s, 0)
dfs(s, start):
    if memoized for start: return cached list
    result = empty list
    if start == length of s: result = [""], return it   (base: empty suffix gives one empty sentence)
    for end from start+1 to length of s:
        word = s[start..end)
        if word is in dictionary:
            for each tail in dfs(s, end):               (all segmentations of remainder)
                add (word, or word + " " + tail) to result
    memoize result for start
    return result
```

```java
class Solution {
    private Map<Integer, List<String>> memo = new HashMap<>();
    private Set<String> words;

    public List<String> wordBreak(String s, List<String> wordDict) {
        words = new HashSet<>(wordDict);
        return dfs(s, 0);
    }
    private List<String> dfs(String s, int start) {             // ← VARIATION: memoized DFS returns se
        if (memo.containsKey(start)) {
            return memo.get(start);
        }
        List<String> result = new ArrayList<>();
        if (start == s.length()) {
            result.add("");
            return result;
        }
        for (int i = start + 1; i <= s.length(); i++) {
            String word = s.substring(start, i);
            if (words.contains(word)) {
                for (String tail : dfs(s, i)) {
                    result.add(tail.isEmpty() ? word : word + " " + tail);
                }
            }
        }
        memo.put(start, result);
        return result;
    }
}
```

Time $O(n^2 \cdot 2^n)$ worst case | **Space** $O(2^n)$

#152 Maximum Product Subarray

Description: Find the contiguous subarray with the largest product. **Intuition:** a negative number flips max and min, so track both; the running max product ending at `i` may come from the prior max or min. **Algorithm:** maintain `maxEnd` and `minEnd`; swap on negative; track global `result`.

Variables: `maxEnd` = maximum product of a subarray ending at index `i`; `minEnd` = minimum (most negative) product ending at `i`; `result` = best product seen overall. **Pseudocode:**

```
maxEnd = minEnd = result = nums[0]
for i from 1 to n-1:
    if nums[i] < 0: swap maxEnd and minEnd   (a negative turns the largest into the smallest)
    maxEnd = max(nums[i], maxEnd * nums[i])  (extend or restart for the max)
    minEnd = min(nums[i], minEnd * nums[i])  (extend or restart for the min)
    result = max(result, maxEnd)
return result
```

```java
class Solution {
    public int maxProduct(int[] nums) {
        int maxEnd = nums[0], minEnd = nums[0], result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] < 0) {                                  // ← VARIATION: negative swaps max/min
                int temp = maxEnd;
                maxEnd = minEnd;
                minEnd = temp;
            }
            maxEnd = Math.max(nums[i], maxEnd * nums[i]);
            minEnd = Math.min(nums[i], minEnd * nums[i]);
            result = Math.max(result, maxEnd);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#174 Dungeon Game

Description: Find the minimum starting health so the knight survives from top-left to bottom-right (each cell adds/subtracts health, must stay ≥ 1). **Intuition:** health needed depends on the future, so fill from the bottom-right: needed entering a cell = max(1, min child need − cell value). **Algorithm:** `dp[i][j]` = min health needed entering `(i,j)`; answer `dp[0][0]`.

Variables: `dp[i][j]` = minimum health the knight must have on entering cell `(i,j)` to survive to the end; `need` = required health before applying this cell; `m`, `n` = grid dimensions. **Pseudocode:**

```
m, n = grid dimensions
make dp of size (m+1) x (n+1), fill with infinity
dp[m][n-1] = dp[m-1][n] = 1   (just past the goal needs 1 health)
for i from m-1 down to 0:       (fill from bottom-right toward start)
    for j from n-1 down to 0:
        need = min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j]   (health for cheaper next cell, minus this ce
        dp[i][j] = max(1, need)                              (must stay at least 1)
return dp[0][0]
```

```java
class Solution {
    public int calculateMinimumHP(int[][] dungeon) {
        int m = dungeon.length, n = dungeon[0].length;
        int[][] dp = new int[m+1][n+1];
        for (int[] row : dp) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }
        dp[m][n-1] = dp[m-1][n] = 1;
        for (int i = m - 1; i >= 0; i--) {                    // ← VARIATION: fill from bottom-right
            for (int j = n - 1; j >= 0; j--) {
                int need = Math.min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j];
                dp[i][j] = Math.max(1, need);                 // ← VARIATION: health must stay ≥ 1
            }
        }
        return dp[0][0];
    }
}
```

Time O(m·n) | **Space** O(m·n)

#188 Best Time to Buy and Sell Stock IV

Description: Maximize profit with at most `k` transactions. **Intuition:** for each allowed transaction count `t`, track best buy and sell; this is #123 generalized to `k` chained state pairs. **Algorithm:** `buy[t]` / `sell[t]` for `t` in `1..k`; answer `sell[k]`.

Variables: `buy[t]` = max profit so far having opened (but not yet closed) the t-th transaction; `sell[t]` = max profit so far having completed t transactions. `buy` initialized to MIN_VALUE (no transaction open yet), `sell` to 0. **Pseudocode:**

```
if no prices or k == 0: return 0
buy[1..k] = -infinity, sell[0..k] = 0
for each price in prices:
    for t from 1 to k:
        buy[t]  = max(buy[t], sell[t-1] - price)   // open t-th by buying now
        sell[t] = max(sell[t], buy[t] + price)     // close t-th by selling now
return sell[k]
```

```java
class Solution {
    public int maxProfit(int k, int[] prices) {
        if (prices.length == 0 || k == 0) {
            return 0;
        }
        int[] buy = new int[k + 1], sell = new int[k + 1];
        Arrays.fill(buy, Integer.MIN_VALUE);
        for (int price : prices) {
            for (int t = 1; t <= k; t++) {                    // ← VARIATION: k chained state pairs
                buy[t] = Math.max(buy[t], sell[t-1] - price);
                sell[t] = Math.max(sell[t], buy[t] + price);
            }
        }
        return sell[k];
    }
}
```

Time O(n·k) | **Space** O(k)

#264 Ugly Number II

Description: Find the nth ugly number (positive integers whose only prime factors are 2, 3, 5). **Intuition:** every ugly number is a previous ugly number times 2, 3, or 5; merge the three multiples with pointers. **Algorithm:** `dp[i]` = i-th ugly number; advance pointers `p2,p3,p5`; answer `dp[n-1]`.

Variables: `dp[i]` = the (i+1)-th ugly number (sorted ascending); `p2/p3/p5` = indices into dp of the next ugly number to be multiplied by 2/3/5. **Pseudocode:**

```
dp[0] = 1
p2 = p3 = p5 = 0
for i from 1 to n-1:
    next2 = dp[p2]*2; next3 = dp[p3]*3; next5 = dp[p5]*5
    dp[i] = min(next2, next3, next5)
    if dp[i] == next2: p2++
    if dp[i] == next3: p3++   // separate ifs dedupe equal candidates
    if dp[i] == next5: p5++
return dp[n-1]
```

```java
class Solution {
    public int nthUglyNumber(int n) {
        int[] dp = new int[n];
        dp[0] = 1;
        int p2 = 0, p3 = 0, p5 = 0;
        for (int i = 1; i < n; i++) {                          // ← VARIATION: merge multiples via poi
            int next2 = dp[p2] * 2, next3 = dp[p3] * 3, next5 = dp[p5] * 5;
            dp[i] = Math.min(next2, Math.min(next3, next5));
            if (dp[i] == next2) p2++;
            if (dp[i] == next3) p3++;
            if (dp[i] == next5) p5++;
        }
        return dp[n-1];
    }
}
```

Time O(n) | **Space** O(n)

#313 Super Ugly Number

Description: Find the nth super ugly number whose prime factors all come from a given `primes` list. **Intuition:** generalize #264 to arbitrary primes — keep one pointer per prime and pick the minimum next candidate. **Algorithm:** `dp[i]` = i-th super ugly; `pointers[j]` for each prime; answer `dp[n-1]`.

Variables: `dp[i]` = the (i+1)-th super ugly number; `pointers[j]` = index into dp of the next super ugly number to multiply by `primes[j]`; `k` = number of primes. **Pseudocode:**

```
dp[0] = 1
pointers[0..k-1] = 0
for i from 1 to n-1:
    min = +infinity
    for each prime j: min = min(min, dp[pointers[j]] * primes[j])
    dp[i] = min
    for each prime j:
        if dp[pointers[j]] * primes[j] == min: pointers[j]++   // advance all that tie
return dp[n-1]
```

```java
class Solution {
    public int nthSuperUglyNumber(int n, int[] primes) {
        int k = primes.length;
        int[] dp = new int[n], pointers = new int[k];
        dp[0] = 1;
        for (int i = 1; i < n; i++) {
            int min = Integer.MAX_VALUE;
            for (int j = 0; j < k; j++) {                        // ← VARIATION: one pointer per prime
                min = Math.min(min, dp[pointers[j]] * primes[j]);
            }
            dp[i] = min;
            for (int j = 0; j < k; j++) {
                if (dp[pointers[j]] * primes[j] == min) {
                    pointers[j]++;
                }
            }
        }
        return dp[n-1];
    }
}
```

Time O(n·k) | **Space** O(n+k)

#337 House Robber III

Description: Houses form a binary tree; you cannot rob two directly-linked houses. Maximize the total. **Intuition:** each node returns two values — best if it is robbed (skip children) vs not robbed (children free to choose). **Algorithm:** `dfs(node)` returns `{notRob, rob}`; answer = `max` of root's pair.

Variables: `dfs(node)` returns a 2-element array `{notRob, rob}`: `[0]` = max money from this subtree if `node` is NOT robbed, `[1]` = max money if `node` IS robbed. **Pseudocode:**

```
rob(root):
    result = dfs(root)
    return max(result[notRob], result[rob])

dfs(node):
    if node == null: return {0, 0}
    left  = dfs(node.left)
    right = dfs(node.right)
    notRob = max(left.notRob, left.rob) + max(right.notRob, right.rob)
    rob    = node.val + left.notRob + right.notRob   // children must be skipped
    return {notRob, rob}
```

```java
class Solution {
    public int rob(TreeNode root) {
        int[] result = dfs(root);
        return Math.max(result[0], result[1]);
    }
    private int[] dfs(TreeNode node) {                          // ← VARIATION: tree DP returns {notRob
        if (node == null) {
            return new int[]{0, 0};
        }
        int[] left = dfs(node.left), right = dfs(node.right);
        int notRob = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);
        int rob = node.val + left[0] + right[0];
        return new int[]{notRob, rob};
    }
}
```

Time O(n) | **Space** O(h)

#343 Integer Break

Description: Break `n` into the sum of at least two positive integers maximizing their product. **Intuition:** for each split `j`, the rest can stay as `i-j` or be broken further (`dp[i-j]`); take the best. **Algorithm:** `dp[i]` = max product of breaking `i` (into ≥2 parts); answer `dp[n]`.

Variables: `dp[i]` = maximum product obtainable by breaking integer `i` into at least two positive parts; `j` = first part, `i-j` = remainder (kept whole or broken further via `dp[i-j]`). **Pseudocode:**

```
dp[1] = 1
for i from 2 to n:
    for j from 1 to i-1:
        dp[i] = max(dp[i], max(j*(i-j), j*dp[i-j]))   // remainder whole vs broken
return dp[n]
```

```java
class Solution {
    public int integerBreak(int n) {
        int[] dp = new int[n + 1];
        dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            for (int j = 1; j < i; j++) {                       // ← VARIATION: split point, break furt
                dp[i] = Math.max(dp[i], Math.max(j * (i - j), j * dp[i - j]));
            }
        }
        return dp[n];
    }
}
```

Time O(n²) | **Space** O(n)

#375 Guess Number Higher or Lower II

Description: Find the minimum amount of money to guarantee a win when guessing a number in `[1, n]`, paying the guessed value on each wrong guess. **Intuition:** for range `[i,j]`, guessing `k` costs `k` plus the worst of the two resulting subranges; minimize over `k`. **Algorithm:** `dp[i][j]` = min worst-case cost for `[i,j]` (interval DP); answer `dp[1][n]`.

Variables: `dp[i][j]` = minimum money guaranteeing a win when the target is somewhere in `[i,j]`; `len` = current interval length; `k` = guessed number; `cost` = `k` + worst of the two resulting subranges. **Pseudocode:**

```
dp sized (n+2)x(n+2), all 0
for len from 2 to n:
    for i from 1 while i+len-1 <= n:
        j = i+len-1
        dp[i][j] = +infinity
        for k from i to j:
            cost = k + max(dp[i][k-1], dp[k+1][j])   // worst-case subrange
            dp[i][j] = min(dp[i][j], cost)
return dp[1][n]
```

```java
class Solution {
    public int getMoneyAmount(int n) {
        int[][] dp = new int[n+2][n+2];
        for (int len = 2; len <= n; len++) {                    // ← VARIATION: interval DP over range
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i; k <= j; k++) {                  // ← VARIATION: guess k, take worst sub
                    int cost = k + Math.max(dp[i][k-1], dp[k+1][j]);
                    dp[i][j] = Math.min(dp[i][j], cost);
                }
            }
        }
        return dp[1][n];
    }
}
```

Time $O(n^3)$ | **Space** $O(n^2)$

#376 Wiggle Subsequence

Description: Find the length of the longest subsequence whose consecutive differences strictly alternate between positive and negative. **Intuition:** track the best subsequence ending with an up-move and with a down-move; each rise extends `down`, each fall extends `up`. **Algorithm:** `up` / `down` running lengths; answer `max(up, down)`.

Variables: `up` = length of longest wiggle subsequence ending with an upward (rising) difference; `down` = same ending with a downward difference. Both start at 1. **Pseudocode:**

```
up = down = 1
for i from 1 to n-1:
    if nums[i] > nums[i-1]: up = down + 1      // rise extends a down-ending seq
    else if nums[i] < nums[i-1]: down = up + 1 // fall extends an up-ending seq
    (equal: change nothing)
return max(up, down)
```

```java
class Solution {
    public int wiggleMaxLength(int[] nums) {
        int up = 1, down = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i-1]) {                          // ← VARIATION: rise extends down-endin
                up = down + 1;
            } else if (nums[i] < nums[i-1]) {
                down = up + 1;
            }
        }
        return Math.max(up, down);
    }
}
```

Time O(n) | **Space** O(1)

#413 Arithmetic Slices

Description: Count the number of arithmetic subarrays (length ≥ 3, constant difference). **Intuition:** if `nums[i]` continues the arithmetic run ending at `i-1`, it adds `dp[i-1]+1` new slices ending at `i`. **Algorithm:** `current` = arithmetic slices ending at `i`; accumulate into `result`.

Variables: `current` = number of arithmetic subarrays ending exactly at index `i`; `result` = running total of all arithmetic slices found. **Pseudocode:**

```
current = 0, result = 0
for i from 2 to n-1:
    if nums[i]-nums[i-1] == nums[i-1]-nums[i-2]:   // run continues
        current++
        result += current
    else:
        current = 0
return result
```

```java
class Solution {
    public int numberOfArithmeticSlices(int[] nums) {
        int current = 0, result = 0;
        for (int i = 2; i < nums.length; i++) {
            if (nums[i] - nums[i-1] == nums[i-1] - nums[i-2]) {  // ← VARIATION: extend arithmetic run
                current++;
                result += current;
            } else {
                current = 0;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#446 Arithmetic Slices II - Subsequence

Description: Count the arithmetic subsequences (length ≥ 3) of the array. **Intuition:** for each pair `(j, i)` with difference `d`, weak slices ending at `i` extend those ending at `j`; promote weak to valid when length reaches 3. **Algorithm:**

`dp[i]` is a map from difference `d` to count of weak arithmetic subsequences ending at `i`; accumulate `result`.

Variables: `dp[i]` = map from common difference `d` to the count of "weak" arithmetic subsequences (length >= 2) ending at index `i`; `d` = `nums[i]-nums[j]`; `countJ` = weak subsequences with diff `d` ending at `j` (each becomes a valid length-3+ slice when extended to `i`); `result` = total valid slices. **Pseudocode:**

```
result = 0
for i from 0 to n-1:
    dp[i] = empty map
    for j from 0 to i-1:
        d = nums[i] - nums[j]                  // use long
        countJ = dp[j].getOrDefault(d, 0)
        result += countJ                       // those weak seqs become valid at i
        dp[i][d] += countJ + 1                 // +1 for the new pair (j,i)
return result
```

```java
class Solution {
    public int numberOfArithmeticSlices(int[] nums) {
        int n = nums.length, result = 0;
        Map<Long, Integer>[] dp = new HashMap[n];              // ← VARIATION: per-index map keyed by
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                long d = (long) nums[i] - nums[j];
                int countJ = dp[j].getOrDefault(d, 0);
                result += countJ;                              // ← VARIATION: weak at j becomes valid
                dp[i].merge(d, countJ + 1, Integer::sum);
            }
        }
        return result;
    }
}
```

Time O(n²) | **Space** O(n²)

#486 Predict the Winner

Description: Two players alternately take a number from either end; decide whether player 1 can win (score ≥ player 2). **Intuition:** on range `[i,j]` the current player maximizes value − opponent's best on the remaining range. **Algorithm:** `dp[i][j]` = best score difference (current minus opponent) for `[i,j]`; answer `dp[0][n-1] >= 0`.

Variables: `dp[i][j]` = best achievable score difference (current player's score minus opponent's) when only `nums[i..j]` remain and it is the current player's turn. **Pseudocode:**

```
for i: dp[i][i] = nums[i]              // single element: take it
for i from n-1 down to 0:
    for j from i+1 to n-1:
        // take left or right; opponent then plays optimally on the rest
        dp[i][j] = max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1])
return dp[0][n-1] >= 0
```

```java
class Solution {
    public boolean predictTheWinner(int[] nums) {
        int n = nums.length;
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            dp[i][i] = nums[i];
        }
        for (int i = n - 1; i >= 0; i--) {                    // ← VARIATION: interval DP, score diff
            for (int j = i + 1; j < n; j++) {
                dp[i][j] = Math.max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1]);
            }
        }
        return dp[0][n-1] >= 0;
    }
}
```

Time O(n²) | **Space** O(n²)

#509 Fibonacci Number

Description: Return the nth Fibonacci number. **Intuition:** each term is the sum of the previous two — the canonical linear 1D recurrence. **Algorithm:** roll two variables `previous` and `current`; answer after `n` steps.

Variables: `previous` = F(i-2), `current` = F(i-1) as the loop builds F(i); rolled forward each step. Conceptually `dp[i] = dp[i−1] + dp[i−2]` reduced to two variables. **Pseudocode:**

```
if n < 2: return n
previous = 0, current = 1          // F(0), F(1)
for i from 2 to n:
    next = previous + current
    previous = current
    current = next
return current
```

```java
class Solution {
    public int fib(int n) {
        if (n < 2) {
            return n;
        }
        int previous = 0, current = 1;
        for (int i = 2; i <= n; i++) {
            int next = previous + current;
            previous = current;
            current = next;
        }
        return current;
    }
}
```

Time O(n) | **Space** O(1)

#552 Student Attendance Record II

Description: Count length-`n` attendance records that are rewardable (at most one 'A', no three consecutive 'L'), modulo 1e9+7. **Intuition:** state = (number of 'A's so far 0/1, trailing 'L' run 0/1/2); each day append P, A, or L respecting limits. **Algorithm:** `dp[a][l]` counts records ending in that state; roll day by day; answer = sum over all states.

Variables: `dp[a][l]` = number of valid records built so far that contain `a` total 'A's (0 or 1) and end with a trailing run of exactly `l` consecutive 'L's (0,1,2); `next` = the same table for the following day; `ways` = current state's count being propagated. **Pseudocode:**

```
dp[0][0] = 1
repeat n days:
    next = zeros
    for a in {0,1}, l in {0,1,2}:
        ways = dp[a][l]; if 0 skip
        next[a][0] += ways                 // append 'P' -> L run resets
        if a == 0: next[1][0] += ways      // append 'A' (only if no A yet)
        if l < 2:  next[a][l+1] += ways    // append 'L' (run stays < 3)
    dp = next  (all mod 1e9+7)
return sum of all dp[a][l] mod
```

```java
class Solution {
    public int checkRecord(int n) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[2][3];                          // ← VARIATION: state = (A count, trail
        dp[0][0] = 1;
        for (int day = 0; day < n; day++) {
            long[][] next = new long[2][3];
            for (int a = 0; a < 2; a++) {
                for (int l = 0; l < 3; l++) {
                    long ways = dp[a][l];
                    if (ways == 0) {
                        continue;
                    }
                    next[a][0] = (next[a][0] + ways) % MOD;     // append 'P' resets L run
                    if (a == 0) {
                        next[1][0] = (next[1][0] + ways) % MOD; // append 'A' (only if none yet)
                    }
                    if (l < 2) {
                        next[a][l+1] = (next[a][l+1] + ways) % MOD; // append 'L' (run < 3)
                    }
                }
            }
            dp = next;
        }
        long result = 0;
        for (int a = 0; a < 2; a++) {
            for (int l = 0; l < 3; l++) {
                result = (result + dp[a][l]) % MOD;
            }
        }
        return (int) result;
    }
}
```

Time O(n) | **Space** O(1)

#576 Out of Boundary Paths

Description: Count paths that move a ball off the grid in exactly `maxMove` steps from a start cell, modulo 1e9+7. **Intuition:** moving off the boundary counts as one path; otherwise spread current-step counts to 4 neighbors for the next step. **Algorithm:** `dp[i][j]` = ways to be at `(i,j)` after `step` moves; sum boundary exits; answer = accumulated `result`.

Variables: `dp[i][j]` = number of ways the ball is at cell `(i,j)` after `step` moves; `next` = same grid for the following step; `result` = accumulated count of paths that have already exited the grid; `dr/dc` = the 4 move directions.
Pseudocode:

```
dp[startRow][startColumn] = 1
result = 0
for step from 0 to maxMove-1:
    next = zeros
    for each cell (i,j) with dp[i][j] != 0:
        for each of 4 directions:
            (ni,nj) = neighbor
            if neighbor off grid: result += dp[i][j]       // exiting = one path
            else: next[ni][nj] += dp[i][j]
    dp = next  (all mod 1e9+7)
return result
```

```java
class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[m][n];
        dp[startRow][startColumn] = 1;
        long result = 0;
        for (int step = 0; step < maxMove; step++) {
            long[][] next = new long[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (dp[i][j] == 0) {
                        continue;
                    }
                    for (int dir = 0; dir < 4; dir++) {         // ← VARIATION: spread to 4 neighbors
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni < 0 || ni >= m || nj < 0 || nj >= n) {
                            result = (result + dp[i][j]) % MOD; // ← VARIATION: stepping off grid = a p
                        } else {
                            next[ni][nj] = (next[ni][nj] + dp[i][j]) % MOD;
                        }
                    }
                }
            }
            dp = next;
        }
        return (int) result;
    }
}
```

Time O(maxMove·m·n) | **Space** O(m·n)

#600 Non-negative Integers without Consecutive Ones

Description: Count integers in `[0, n]` whose binary representation has no two adjacent 1 bits. **Intuition:** scan bits high-to-low; precompute Fibonacci-style counts of valid suffixes, adding them when a free bit is `1`, and stop early if two consecutive 1s appear in `n`. **Algorithm:** `fib[i]` = valid numbers with `i` free bits; walk bits of `n`; answer = accumulated `result` (+1 for `n` itself if valid).

Variables: `fib[i]` = count of binary strings of length `i` with no two adjacent 1s (Fibonacci: `fib[i]=fib[i-1]+fib[i-2]`); `result` = accumulated count of valid numbers strictly below `n` so far; `previousBit` = the previous (higher) bit of `n` seen, to detect two consecutive 1s. **Pseudocode:**

```
build fib[0..31]: fib[0]=1, fib[1]=2, fib[i]=fib[i-1]+fib[i-2]
result = 0, previousBit = 0
for bit i from 30 down to 0:
    if bit i of n is 1:
        result += fib[i]                // count numbers having 0 at this bit
        if previousBit == 1: return result   // n itself has adjacent 1s; stop
        previousBit = 1
    else:
        previousBit = 0
return result + 1                       // n itself is valid
```

```java
class Solution {
    public int findIntegers(int n) {
        int[] fib = new int[32];                                // ← VARIATION: fib[i] = valid i-bit st
        fib[0] = 1; fib[1] = 2;
        for (int i = 2; i < 32; i++) {
            fib[i] = fib[i-1] + fib[i-2];
        }
        int result = 0, previousBit = 0;
        for (int i = 30; i >= 0; i--) {
            if ((n & (1 << i)) != 0) {                          // current bit is 1
                result += fib[i];                               // ← VARIATION: count numbers with 0 he
                if (previousBit == 1) {
                    return result;                              // ← VARIATION: two adjacent 1s in n, s
                }
                previousBit = 1;
            } else {
                previousBit = 0;
            }
        }
        return result + 1;                                      // include n itself
    }
}
```

Time O(1) (32 bits) | **Space** O(1)

#629 K Inverse Pairs Array

Description: Count permutations of `1..n` with exactly `k` inverse pairs, modulo 1e9+7. **Intuition:** inserting `n` into a permutation of `1..n-1` adds `0..n-1` inversions; this gives a sliding-window sum over the previous row. **Algorithm:** `dp[i][j]` = permutations of `i` numbers with `j` inversions, using prefix sums; answer `dp[n][k]`.

Variables: `dp[i][j]` = number of permutations of `1..i` having exactly `j` inverse pairs; `prefix[j+1]` = prefix sum of row `dp[i-1]` over `[0..j]`, used to sum the window of allowed contributions; `lo` = window's lower index `max(0, j-(i-1))`. **Pseudocode:**

```
dp[0][0] = 1
for i from 1 to n:
    prefix[0]=0; prefix[j+1] = prefix[j] + dp[i-1][j]   for j in 0..k
    for j from 0 to k:
        lo = max(0, j-(i-1))                // inserting i adds 0..i-1 inversions
        dp[i][j] = prefix[j+1] - prefix[lo]  (mod)
return dp[n][k]
```

```java
class Solution {
    public int kInversePairs(int n, int k) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[n+1][k+1];
        dp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            long[] prefix = new long[k+2];
            for (int j = 0; j <= k; j++) {
                prefix[j+1] = (prefix[j] + dp[i-1][j]) % MOD;   // ← VARIATION: prefix sums of previous
            }
            for (int j = 0; j <= k; j++) {                      // window [j-(i-1), j] of previous row
                int lo = Math.max(0, j - (i - 1));
                dp[i][j] = (prefix[j+1] - prefix[lo] + MOD) % MOD;
            }
        }
        return (int) dp[n][k];
    }
}
```

Time O(n·k) | **Space** O(n·k)

#638 Shopping Offers

Description: Buy exactly `needs[i]` of each item at minimum cost, given unit prices and special bundle offers (each usable multiple times). **Intuition:** treat the remaining needs vector as the state; either buy everything at unit price, or apply an affordable offer and recurse. **Algorithm:** memoized DFS over the needs list; answer `dfs(needs)`.

Variables: state = the `needs` vector (remaining quantity required of each item); `memo` maps a needs vector to its min cost; `best` = best cost for the current needs (baseline = buying everything at unit price); each `offer` reduces needs by its bundle and adds its price. **Pseudocode:**

```
dfs(needs):
    if memo has needs: return it
    best = sum over items of needs[i] * price[i]      // buy all at unit price
    for each offer:
        remaining = needs - offer (per item); skip offer if any goes negative
        if valid: best = min(best, offer.price + dfs(remaining))
    memo[needs] = best
    return best
```

```java
class Solution {
    private Map<List<Integer>, Integer> memo = new HashMap<>();

    public int shoppingOffers(List<Integer> price, List<List<Integer>> special, List<Integer> needs) {
        return dfs(price, special, needs);
    }
    private int dfs(List<Integer> price, List<List<Integer>> special, List<Integer> needs) {  // ← VARI
        if (memo.containsKey(needs)) {
            return memo.get(needs);
        }
        int n = price.size(), best = 0;
        for (int i = 0; i < n; i++) {
            best += needs.get(i) * price.get(i);              // baseline: all at unit price
        }
        for (List<Integer> offer : special) {
            List<Integer> remaining = new ArrayList<>();
            boolean valid = true;
            for (int i = 0; i < n; i++) {
                int left = needs.get(i) - offer.get(i);
                if (left < 0) {                                // ← VARIATION: offer must not overbuy
                    valid = false;
                    break;
                }
                remaining.add(left);
            }
            if (valid) {
                best = Math.min(best, offer.get(n) + dfs(price, special, remaining));
            }
        }
        memo.put(needs, best);
        return best;
    }
}
```

Time $O(\text{offers} \cdot \prod \text{needs}_i)$ | **Space** $O(\prod \text{needs}_i)$

#639 Decode Ways II

Description: Count decodings of a digit string where `*` is a wildcard for digits 1-9, modulo 1e9+7. **Intuition:** extend #91 — each position can decode one char (count 1-9 options) or two chars (count valid 10-26 combinations); `*` multiplies the options. **Algorithm:** `dp[i]` = decodings of `s[0..i)`; answer `dp[n]`.

Variables: `dp[i]` = number of ways to decode the prefix `s[0..i)`; `oneDigit(c)` = number of single-digit decodings for char `c` (`*`->9, '0'->0, else 1); `twoDigit(a,b)` = number of two-digit decodings in 10..26 for the pair (handling `*` wildcards). **Pseudocode:**

```
dp[0] = 1
dp[1] = oneDigit(s[0])
for i from 2 to n:
    current = s[i-1], prev = s[i-2]
    dp[i] += dp[i-1] * oneDigit(current)        // decode last char alone
    dp[i] += dp[i-2] * twoDigit(prev, current)  // decode last two chars
    (mod 1e9+7)
return dp[n]
```

```java
class Solution {
    public int numDecodings(String s) {
        int MOD = 1_000_000_007, n = s.length();
        long[] dp = new long[n + 1];
        dp[0] = 1;
        dp[1] = oneDigit(s.charAt(0));
        for (int i = 2; i <= n; i++) {
            char current = s.charAt(i-1), prev = s.charAt(i-2);
            dp[i] = (dp[i] + dp[i-1] * oneDigit(current)) % MOD; // ← VARIATION: single char, count opt
            dp[i] = (dp[i] + dp[i-2] * twoDigit(prev, current)) % MOD; // ← VARIATION: pair, count 10-2
        }
        return (int) dp[n];
    }
    private long oneDigit(char c) {
        if (c == '*') return 9;                                  // 1-9
        return c == '0' ? 0 : 1;
    }
    private long twoDigit(char a, char b) {
        if (a == '*' && b == '*') return 15;                     // 11-19, 21-26
        if (a == '*') {                                           // *X
            return b <= '6' ? 2 : 1;                             // 1X always, 2X only if X<=6
        }
        if (b == '*') {                                           // X*
            if (a == '1') return 9;                              // 11-19
            if (a == '2') return 6;                              // 21-26
            return 0;
        }
        int value = (a - '0') * 10 + (b - '0');
        return value >= 10 && value <= 26 ? 1 : 0;
    }
}
```

Time O(n) | **Space** O(n)

#650 2 Keys Keyboard

Description: Starting with one 'A', using only Copy-All and Paste, return the minimum operations to obtain `n` 'A's. **Intuition:** the answer is the sum of `n`'s prime factors — building `i` from a divisor `j` costs `i/j` operations. **Algorithm:** `dp[i]` = min operations to reach `i` characters; for each divisor `j`, `dp[i] = dp[j] + i/j`; answer `dp[n]`.

Variables: `dp[i]` = minimum operations to produce exactly `i` characters; `j` = largest proper divisor of `i` (building `i` from `j` costs one Copy-All + `(i/j - 1)` Pastes = `i/j` ops). **Pseudocode:**

```
for i from 2 to n:
    dp[i] = i                       // upper bound (copy once, paste i-1 times)
    for j from i/2 down to 1:
        if i % j == 0:
            dp[i] = dp[j] + i/j     // largest divisor gives the minimum
            break
return dp[n]
```

```java
class Solution {
    public int minSteps(int n) {
        int[] dp = new int[n + 1];
        for (int i = 2; i <= n; i++) {
            dp[i] = i;                                          // worst case: copy then paste i-1 time
            for (int j = i / 2; j >= 1; j--) {                  // ← VARIATION: build i from divisor j
                if (i % j == 0) {
                    dp[i] = dp[j] + i / j;
                    break;                                      // largest divisor gives the min
                }
            }
        }
        return dp[n];
    }
}
```

Time O(n√n) | **Space** O(n)

#673 Number of Longest Increasing Subsequence

Description: Count the number of longest strictly increasing subsequences. **Intuition:** alongside LIS length, track how many ways achieve it; a longer extension resets the count, an equal-length extension accumulates it. **Algorithm:** `length[i]` and `count[i]` for LIS ending at `i`; sum counts of maximum-length endings; answer `result`.

Variables: `length[i]` = length of the longest increasing subsequence ending at index `i`; `count[i]` = number of LIS of that length ending at `i`; `maxLen` = global max LIS length seen; `result` = total count of LIS achieving `maxLen`.
Pseudocode:

```
for i from 0 to n-1:
    length[i] = 1; count[i] = 1
    for j from 0 to i-1 with nums[j] < nums[i]:
        if length[j]+1 > length[i]:        // found longer -> reset count
            length[i] = length[j]+1; count[i] = count[j]
        else if length[j]+1 == length[i]:  // equal length -> add count
            count[i] += count[j]
    if length[i] > maxLen: maxLen = length[i]; result = count[i]
    else if length[i] == maxLen: result += count[i]
return result
```

```java
class Solution {
    public int findNumberOfLIS(int[] nums) {
        int n = nums.length, maxLen = 0, result = 0;
        int[] length = new int[n], count = new int[n];
        for (int i = 0; i < n; i++) {
            length[i] = 1; count[i] = 1;
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    if (length[j] + 1 > length[i]) {            // ← VARIATION: longer extension resets
                        length[i] = length[j] + 1;
                        count[i] = count[j];
                    } else if (length[j] + 1 == length[i]) {    // ← VARIATION: equal length accumulate
                        count[i] += count[j];
                    }
                }
            }
            if (length[i] > maxLen) {
                maxLen = length[i];
                result = count[i];
            } else if (length[i] == maxLen) {
                result += count[i];
            }
        }
        return result;
    }
}
```

Time O(n^2) | **Space** O(n)

#674 Longest Continuous Increasing Subsequence

Description: Find the length of the longest continuous (contiguous) strictly increasing subarray. **Intuition:** extend the current run while values rise, otherwise restart; a simpler linear scan of #300. **Algorithm:** `current` run length; track `result`.

Variables: `current` = length of the current contiguous strictly-increasing run ending at `i`; `result` = longest such run seen. **Pseudocode:**

```
current = 1, result = 1
for i from 1 to n-1:
    if nums[i] > nums[i-1]: current++     // run continues
    else: current = 1                    // reset on non-increase
    result = max(result, current)
return result
```

```java
class Solution {
    public int findLengthOfLCIS(int[] nums) {
        int current = 1, result = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i-1]) {                          // ← VARIATION: contiguous run, reset o
                current++;
            } else {
                current = 1;
            }
            result = Math.max(result, current);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#688 Knight Probability in Chessboard

Description: Return the probability that a knight remains on an `n×n` board after exactly `k` moves from a start cell, each move chosen uniformly among 8. **Intuition:** spread the probability at each cell to its 8 knight-move targets per step; off-board moves lose probability. **Algorithm:** `dp[i][j]` = probability of being at `(i,j)` after `step` moves; answer = sum of final grid.

Variables: `dp[i][j]` = probability the knight is on cell `(i,j)` after `step` moves; `next` = same grid for the following step; `dr/dc` = the 8 knight-move offsets; each move contributes `dp[i][j]/8`. **Pseudocode:**

```
dp[row][column] = 1.0
for step from 0 to k-1:
    next = zeros
    for each cell (i,j) with dp[i][j] != 0:
        for each of 8 knight moves to (ni,nj):
            if (ni,nj) on board: next[ni][nj] += dp[i][j] / 8.0
    dp = next
return sum of all dp[i][j]
```

```java
class Solution {
    int[] dr = {2,2,-2,-2,1,1,-1,-1}, dc = {1,-1,1,-1,2,-2,2,-2};

    public double knightProbability(int n, int k, int row, int column) {
        double[][] dp = new double[n][n];
        dp[row][column] = 1.0;
        for (int step = 0; step < k; step++) {
            double[][] next = new double[n][n];
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dp[i][j] == 0) {
                        continue;
                    }
                    for (int dir = 0; dir < 8; dir++) {         // ← VARIATION: 8 knight moves, each pr
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
                            next[ni][nj] += dp[i][j] / 8.0;
                        }
                    }
                }
            }
            dp = next;
        }
        double result = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                result += dp[i][j];
            }
        }
        return result;
    }
}
```

Time O(k·n²) | **Space** O(n²)

#718 Maximum Length of Repeated Subarray

Description: Find the length of the longest subarray common to two arrays (contiguous). **Intuition:** unlike LCS, the match must be contiguous, so a mismatch resets the running length to 0. **Algorithm:** `dp[i][j]` = length of common suffix ending at `nums1[i-1]`, `nums2[j-1]`; track global `result`.

Variables: `dp[i][j]` = length of the longest common contiguous subarray ending exactly at `nums1[i-1]` and `nums2[j-1]`; `result` = global maximum over all cells. **Pseudocode:**

```
for i from 1 to m:
    for j from 1 to n:
        if nums1[i-1] == nums2[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1     // extend match; mismatch leaves 0
            result = max(result, dp[i][j])
return result
```

```java
class Solution {
    public int findLength(int[] nums1, int[] nums2) {
        int m = nums1.length, n = nums2.length, result = 0;
        int[][] dp = new int[m+1][n+1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (nums1[i-1] == nums2[j-1]) {
                    dp[i][j] = dp[i-1][j-1] + 1;                // ← VARIATION: contiguous, no carry on
                    result = Math.max(result, dp[i][j]);
                }
            }
        }
        return result;
    }
}
```

Time O(m·n) | **Space** O(m·n)

#740 Delete and Earn

Description: Deleting `nums[i]` earns its value but removes all copies of `nums[i]−1` and `nums[i]+1`; maximize earnings. **Intuition:** bucket points by value, then it becomes House Robber over the value line — you cannot take adjacent values. **Algorithm:** `points[v]` = total points for value `v`; `dp[v]` = max earnings up to value `v`; answer `dp[max]`.

Variables: `points[v]` = total points earned from deleting all copies of value `v` (= `v * count`); `dp[v]` = max earnings considering only values `0..v` (House Robber over the value line, since taking `v` forbids `v−1`); `max` = largest value present. **Pseudocode:**

```
max = maximum value in nums
points[v] += v for each num=v       // bucket points by value
dp[0] = 0
dp[1] = points[1]
for v from 2 to max:
    dp[v] = max(dp[v−1], dp[v−2] + points[v])   // skip v, or take v + dp[v−2]
return dp[max]
```

```java
class Solution {
    public int deleteAndEarn(int[] nums) {
        int max = 0;
        for (int num : nums) {
            max = Math.max(max, num);
        }
        int[] points = new int[max + 1];                        // ← VARIATION: bucket points by value
        for (int num : nums) {
            points[num] += num;
        }
        int[] dp = new int[max + 1];
        dp[0] = 0;
        dp[1] = points[1];
        for (int v = 2; v <= max; v++) {                        // ← VARIATION: House Robber over value
            dp[v] = Math.max(dp[v−1], dp[v−2] + points[v]);
        }
        return dp[max];
    }
}
```

Time O(n + max) | **Space** O(max)

#873 Length of Longest Fibonacci Subsequence

Description: Given a strictly increasing array, find the length of the longest Fibonacci-like subsequence (each term = sum of the previous two). **Intuition:** a fib-like subseq is determined by its last two elements `(j, i)`; look up whether the required predecessor `arr[i]−arr[j]` exists earlier. **Algorithm:** `dp[j][i]` = length of fib seq ending with `arr[j], arr[i]`; answer = max found (≥ 3).

Variables: `dp[j][i]` = length of the longest Fibonacci-like subsequence whose last two elements are `arr[j]` then `arr[i]` (j < i); `index` = map from value to its array index; `needed` = the required predecessor `arr[i]−arr[j]`; `result` = best length >= 3. **Pseudocode:**

```
index = map value -> its index
for i from 0 to n-1:
    for j from 0 to i-1:
        needed = arr[i] - arr[j]
        k = index.get(needed)
        if k exists and k < j: dp[j][i] = dp[k][j] + 1   // extend chain ...k,j,i
        else: dp[j][i] = 2                                // seed pair (j,i)
        if dp[j][i] > 2: result = max(result, dp[j][i])
return result
```

```java
class Solution {
    public int lenLongestFibSubseq(int[] arr) {
        int n = arr.length, result = 0;
        Map<Integer, Integer> index = new HashMap<>();
        for (int i = 0; i < n; i++) {
            index.put(arr[i], i);
        }
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                int needed = arr[i] - arr[j];                    // ← VARIATION: predecessor before arr[
                Integer k = index.get(needed);
                if (k != null && k < j) {
                    dp[j][i] = dp[k][j] + 1;
                } else {
                    dp[j][i] = 2;                                // seed pair
                }
                if (dp[j][i] > 2) {
                    result = Math.max(result, dp[j][i]);
                }
            }
        }
        return result;
    }
}
```

Time O(n²) | **Space** O(n²)

#887 Super Egg Drop

Description: With `k` eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case. **Intuition:** invert the problem — `dp[m][k]` = highest floor count solvable with `m` moves and `k` eggs; a move either breaks (test below) or survives (test above), plus the current floor. **Algorithm:** grow `m` until `dp[m][k] >= n`; answer = that `m`.

Variables: `dp[m][eggs]` = the maximum number of floors whose critical floor can be determined with `m` moves and `eggs` eggs; `m` = move count, grown until `dp[m][k] >= n`. **Pseudocode:**

```
m = 0
while dp[m][k] < n:
    m++
    for eggs from 1 to k:
        // drop one egg: break -> dp[m-1][eggs-1] floors below,
        //               survive -> dp[m-1][eggs] floors above, +1 current floor
        dp[m][eggs] = dp[m-1][eggs-1] + dp[m-1][eggs] + 1
return m
```

```java
class Solution {
    public int superEggDrop(int k, int n) {
        int[][] dp = new int[n+1][k+1];                          // ← VARIATION: dp[m][k] = floors solva
        int m = 0;
        while (dp[m][k] < n) {
            m++;
            for (int eggs = 1; eggs <= k; eggs++) {
                dp[m][eggs] = dp[m-1][eggs-1] + dp[m-1][eggs] + 1;  // break + survive + current floor
            }
        }
        return m;
    }
}
```

Time O(k·log n) | **Space** O(n·k)

#918 Maximum Sum Circular Subarray

Description: Find the maximum subarray sum where the array is circular (subarray may wrap around). **Intuition:** the answer is either a normal max subarray (Kadane) or the total minus the minimum subarray (wrap); guard the all-negative case. **Algorithm:** track max and min running subarrays; answer = `max(maxSum, total - minSum)` unless all negative.

Variables: `curMax` = Kadane running max-subarray-sum ending here; `maxSum` = best non-wrapping subarray sum; `curMin` / `minSum` = same for minimum subarray (the wrap answer is `total - minSum`); `total` = sum of all elements. **Pseudocode:**

```
total=0; maxSum=curMax=nums[0]? (curMax starts 0); minSum analog
for each num:
    curMax = max(curMax + num, num);  maxSum = max(maxSum, curMax)
    curMin = min(curMin + num, num);  minSum = min(minSum, curMin)
    total += num
if maxSum < 0: return maxSum            // all negative: wrap would be empty
return max(maxSum, total - minSum)      // non-wrap vs wrap (total minus min)
```

```java
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0, maxSum = nums[0], curMax = 0, minSum = nums[0], curMin = 0;
        for (int num : nums) {
            curMax = Math.max(curMax + num, num);
            maxSum = Math.max(maxSum, curMax);
            curMin = Math.min(curMin + num, num);               // ← VARIATION: track min subarray for
            minSum = Math.min(minSum, curMin);
            total += num;
        }
        if (maxSum < 0) {                                       // ← VARIATION: all negative, wrap inva
            return maxSum;
        }
        return Math.max(maxSum, total - minSum);
    }
}
```

Time O(n) | **Space** O(1)

#935 Knight Dialer

Description: Count distinct phone numbers of length `n` dialable by knight moves on a phone keypad, modulo 1e9+7.
Intuition: from each digit a knight can reach a fixed set of digits; spread counts across the adjacency map each step.
Algorithm: `dp[d]` = count of numbers of current length ending at digit `d`; answer = sum after `n` steps.

Variables: `dp[d]` = count of valid numbers of the current length ending at digit `d`; `next` = same array for length+1; `moves[d]` = digits reachable by a knight move from `d`. **Pseudocode:**

```
moves[d] = knight-reachable digits per keypad digit
dp[0..9] = 1                          // length 1
for step from 1 to n-1:
    next = zeros
    for d from 0 to 9:
        for target in moves[d]: next[target] += dp[d]
    dp = next  (mod 1e9+7)
return sum of dp mod
```

```java
class Solution {
    public int knightDialer(int n) {
        int MOD = 1_000_000_007;
        int[][] moves = {                                    // ← VARIATION: knight adjacency per di
            {4,6},{6,8},{7,9},{4,8},{0,3,9},{},
            {0,1,7},{2,6},{1,3},{2,4}
        };
        long[] dp = new long[10];
        Arrays.fill(dp, 1);                                  // length 1: each digit is one number
        for (int step = 1; step < n; step++) {
            long[] next = new long[10];
            for (int d = 0; d < 10; d++) {
                for (int target : moves[d]) {
                    next[target] = (next[target] + dp[d]) % MOD;
                }
            }
            dp = next;
        }
        long result = 0;
        for (long count : dp) {
            result = (result + count) % MOD;
        }
        return (int) result;
    }
}
```

Time O(n) | **Space** O(1)

#983 Minimum Cost For Tickets

Description: Given travel days and the cost of 1-day, 7-day, and 30-day passes, return the minimum cost to cover all travel days. **Intuition:** on a travel day choose the cheapest pass covering it by looking back 1, 7, or 30 days; non-travel days inherit the prior cost. **Algorithm:** `dp[i]` = min cost through day `i`; answer `dp[lastDay]`.

Variables: `dp[i]` = minimum cost to cover all travel up through day `i`; `travel[i]` = whether day `i` is a travel day; `costs[0/1/2]` = price of 1/7/30-day passes; `last` = final travel day. **Pseudocode:**

```
mark travel[day] = true for each travel day
for i from 1 to last:
    if not travel[i]: dp[i] = dp[i-1]            // no travel: carry cost
    else: dp[i] = min(dp[i-1] + costs[0],        // 1-day pass
                      dp[max(0,i-7)] + costs[1],  // 7-day pass
                      dp[max(0,i-30)] + costs[2]) // 30-day pass
return dp[last]
```

```java
class Solution {
    public int mincostTickets(int[] days, int[] costs) {
        int last = days[days.length - 1];
        boolean[] travel = new boolean[last + 1];
        for (int day : days) {
            travel[day] = true;
        }
        int[] dp = new int[last + 1];
        for (int i = 1; i <= last; i++) {
            if (!travel[i]) {                                   // ← VARIATION: non-travel day inherits
                dp[i] = dp[i-1];
            } else {
                dp[i] = Math.min(dp[i-1] + costs[0],            // ← VARIATION: look back 1/7/30 days
                        Math.min(dp[Math.max(0, i-7)] + costs[1],
                                 dp[Math.max(0, i-30)] + costs[2]));
            }
        }
        return dp[last];
    }
}
```

Time O(lastDay) | **Space** O(lastDay)

#1027 Longest Arithmetic Subsequence

Description: Find the length of the longest arithmetic subsequence (constant difference). **Intuition:** for each pair `(j, i)`, the arithmetic subseq ending at `i` with difference `d` extends the one ending at `j`. **Algorithm:** `dp[i]` maps difference `d` to subseq length ending at `i`; track global `result`.

Variables: `dp[i]` = map from common difference `d` to the length of the longest arithmetic subsequence with that difference ending at index `i`; `d = nums[i]-nums[j]`; `result` = global longest. **Pseudocode:**

```
for i from 0 to n-1:
    dp[i] = empty map
    for j from 0 to i-1:
        d = nums[i] - nums[j]
        length = dp[j].getOrDefault(d, 1) + 1   // extend chain ending at j
        dp[i][d] = length
        result = max(result, length)
return result
```

```java
class Solution {
    public int longestArithSeqLength(int[] nums) {
        int n = nums.length, result = 0;
        Map<Integer, Integer>[] dp = new HashMap[n];            // ← VARIATION: per-index map keyed by
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                int d = nums[i] - nums[j];
                int length = dp[j].getOrDefault(d, 1) + 1;
                dp[i].put(d, length);
                result = Math.max(result, length);
            }
        }
        return result;
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#1048 Longest String Chain

Description: Find the longest chain of words where each word is a predecessor of the next (insert exactly one char). **Intuition:** sort by length; for each word, try removing each character to find a shorter predecessor and extend its best chain. **Algorithm:** `dp` maps word to longest chain ending at it; answer = max chain length.

Variables: `dp` = map from a word to the length of the longest word chain ending at that word; `best` = best chain length for the current word; `predecessor` = the word with one character removed; `result` = global max. **Pseudocode:**

```
sort words by ascending length
for each word (shortest first):
    best = 1
    for each position i in word:
        predecessor = word with char i removed
        if dp has predecessor: best = max(best, dp[predecessor] + 1)
    dp[word] = best
    result = max(result, best)
return result
```

```java
class Solution {
    public int longestStrChain(String[] words) {
        Arrays.sort(words, (a, b) -> a.length() - b.length());  // ← VARIATION: process shortest first
        Map<String, Integer> dp = new HashMap<>();
        int result = 0;
        for (String word : words) {
            int best = 1;
            for (int i = 0; i < word.length(); i++) {           // ← VARIATION: drop one char to find p
                StringBuilder builder = new StringBuilder(word);
                builder.deleteCharAt(i);
                String predecessor = builder.toString();
                if (dp.containsKey(predecessor)) {
                    best = Math.max(best, dp.get(predecessor) + 1);
                }
            }
            dp.put(word, best);
            result = Math.max(result, best);
        }
        return result;
    }
}
```

Time $O(n \cdot L^2)$ | **Space** $O(n)$

#1137 N-th Tribonacci Number

Description: Return the nth Tribonacci number (each term is the sum of the previous three). **Intuition:** the linear 1D recurrence with three rolling variables instead of two. **Algorithm:** roll `a`, `b`, `c`; answer after `n` steps.

Variables: `a, b, c` = the three rolling Tribonacci terms T(i-3), T(i-2), T(i-1) used to compute T(i); rolled forward each step. Conceptually `dp[i]=dp[i−1]+dp[i−2]+dp[i−3]`. **Pseudocode:**

```
if n == 0: return 0
if n <= 2: return 1
a=0, b=1, c=1                       // T0, T1, T2
for i from 3 to n:
    next = a + b + c
    a = b; b = c; c = next
return c
```

```
class Solution {
    public int tribonacci(int n) {
        if (n == 0) {
            return 0;
        }
        if (n <= 2) {
            return 1;
        }
        int a = 0, b = 1, c = 1;
        for (int i = 3; i <= n; i++) {
            int next = a + b + c;                              // ← VARIATION: sum of previous three
            a = b;
            b = c;
            c = next;
        }
        return c;
    }
}
```

Time O(n) | **Space** O(1)

#1216 Valid Palindrome III

Description: Return whether `s` becomes a palindrome after removing at most `k` characters. **Intuition:** the minimum removals to make `s` a palindrome equals `n − LPS(s)`; check whether that is `≤ k`. **Algorithm:** interval DP for longest palindromic subsequence; answer `n − dp[0][n−1] <= k`.

Variables: `dp[i][j]` = length of the longest palindromic subsequence within `s[i..j]`; the minimum deletions to make `s` a palindrome is `n − dp[0][n−1]`. **Pseudocode:**

```
for i from n−1 down to 0:
    dp[i][i] = 1
    for j from i+1 to n−1:
        if s[i] == s[j]: dp[i][j] = dp[i+1][j−1] + 2
        else: dp[i][j] = max(dp[i+1][j], dp[i][j−1])
return n − dp[0][n−1] <= k          // deletions needed <= k
```

```java
class Solution {
    public boolean isValidPalindrome(String s, int k) {
        int n = s.length();
        int[][] dp = new int[n][n];                          // dp[i][j] = LPS in s[i..j]
        for (int i = n - 1; i >= 0; i--) {                   // ← VARIATION: interval DP for LPS
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return n - dp[0][n-1] <= k;                          // ← VARIATION: removals = n - LPS
    }
}
```

Time O(n²) | **Space** O(n²)

#1218 Longest Arithmetic Subsequence of Given Difference

Description: Find the longest arithmetic subsequence with a fixed difference `difference`. **Intuition:** with a known difference, the predecessor of `num` must be `num - difference`; one hashmap pass suffices. **Algorithm:** `dp` maps value to longest chain ending at it; answer = max length.

Variables: `dp` = map from a value to the length of the longest arithmetic subsequence (fixed `difference`) ending at that value; predecessor of `num` is `num - difference`; `result` = global max. **Pseudocode:**

```
for each num in arr:
    length = dp.getOrDefault(num - difference, 0) + 1   // extend chain
    dp[num] = length
    result = max(result, length)
return result
```

```java
class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        Map<Integer, Integer> dp = new HashMap<>();          // ← VARIATION: fixed difference, value
        int result = 0;
        for (int num : arr) {
            int length = dp.getOrDefault(num - difference, 0) + 1;
            dp.put(num, length);
            result = Math.max(result, length);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#1269 Number of Ways to Stay in the Same Place After Some Steps

Description: Count the ways to return to index 0 after `steps` moves (stay, left, or right) without leaving an array of size `arrLen`, modulo 1e9+7. **Intuition:** position can never exceed `steps/2`, so cap the reachable range; each step a position spreads to stay/left/right. **Algorithm:** `dp[i]` = ways to be at index `i` after `step` moves; answer `dp[0]`.

Variables: `dp[i]` = number of ways to be at array index `i` after `step` moves; `next` = same array for the following step; `maxPos` = highest reachable index, capped at `min(arrLen−1, steps/2)`. **Pseudocode:**

```
maxPos = min(arrLen−1, steps/2)
dp[0] = 1
for step from 0 to steps−1:
    next = zeros
    for i from 0 to maxPos with dp[i] != 0:
        next[i]   += dp[i]                // stay
        if i > 0:      next[i−1] += dp[i]  // move left
        if i < maxPos: next[i+1] += dp[i]  // move right
    dp = next  (mod 1e9+7)
return dp[0]
```

```java
class Solution {
    public int numWays(int steps, int arrLen) {
        int MOD = 1_000_000_007;
        int maxPos = Math.min(arrLen − 1, steps / 2);          // ← VARIATION: cap reachable positions
        long[] dp = new long[maxPos + 1];
        dp[0] = 1;
        for (int step = 0; step < steps; step++) {
            long[] next = new long[maxPos + 1];
            for (int i = 0; i <= maxPos; i++) {
                if (dp[i] == 0) {
                    continue;
                }
                next[i] = (next[i] + dp[i]) % MOD;              // stay
                if (i > 0) {
                    next[i−1] = (next[i−1] + dp[i]) % MOD;      // left
                }
                if (i < maxPos) {
                    next[i+1] = (next[i+1] + dp[i]) % MOD;      // right
                }
            }
            dp = next;
        }
        return (int) dp[0];
    }
}
```

Time O(steps·min(arrLen, steps)) | **Space** O(min(arrLen, steps))

#1395 Count Number of Teams

Description: Count triplets `(i, j, k)` with `i < j < k` whose ratings are strictly increasing or strictly decreasing. **Intuition:** fix the middle soldier `j`; the answer is its number of smaller-before × larger-after, plus the mirror case. **Algorithm:** for each `j` count smaller/larger on both sides; accumulate `result`.

Variables: for each middle index `j`: `lessLeft` / `greaterLeft` = counts of ratings smaller/larger before `j`; `lessRight` / `greaterRight` = counts smaller/larger after `j`; `result` = total valid teams. **Pseudocode:**

```
for j from 0 to n-1:                        // fix the middle soldier
    count lessLeft, greaterLeft over i < j
    count lessRight, greaterRight over k > j
    // increasing: small-left * large-right; decreasing: large-left * small-right
    result += lessLeft*greaterRight + greaterLeft*lessRight
return result
```

```java
class Solution {
    public int numTeams(int[] rating) {
        int n = rating.length, result = 0;
        for (int j = 0; j < n; j++) {                          // ← VARIATION: fix middle soldier
            int lessLeft = 0, greaterLeft = 0, lessRight = 0, greaterRight = 0;
            for (int i = 0; i < j; i++) {
                if (rating[i] < rating[j]) lessLeft++;
                else if (rating[i] > rating[j]) greaterLeft++;
            }
            for (int k = j + 1; k < n; k++) {
                if (rating[k] < rating[j]) lessRight++;
                else if (rating[k] > rating[j]) greaterRight++;
            }
            result += lessLeft * greaterRight + greaterLeft * lessRight;  // ← VARIATION: increasing +
        }
        return result;
    }
}
```

Time O(n^2) | **Space** O(1)

#1567 Maximum Length of Subarray With Positive Product

Description: Find the length of the longest subarray whose product is positive. **Intuition:** track the lengths of positive-product and negative-product runs ending at `i`; a negative number swaps them, a zero resets both. **Algorithm:** `positive` / `negative` running lengths; track `result`.

Variables: `positive` = length of the longest subarray ending at `i` with a positive product; `negative` = length of the longest such subarray with a negative product; both reset at a zero; a negative number swaps their roles; `result` = best `positive` seen. **Pseudocode:**

```
positive = negative = result = 0
for each num:
    if num == 0: positive = 0; negative = 0
    else if num > 0:
        positive++
        negative = negative > 0 ? negative + 1 : 0
    else:  // negative number swaps positive/negative runs
        newPositive = negative > 0 ? negative + 1 : 0
        newNegative = positive + 1
        positive = newPositive; negative = newNegative
    result = max(result, positive)
return result
```

```java
class Solution {
    public int getMaxLen(int[] nums) {
        int positive = 0, negative = 0, result = 0;
        for (int num : nums) {
            if (num == 0) {                                    // ← VARIATION: zero resets both runs
                positive = 0;
                negative = 0;
            } else if (num > 0) {
                positive++;
                negative = negative > 0 ? negative + 1 : 0;
            } else {                                           // ← VARIATION: negative swaps pos/neg
                int newPositive = negative > 0 ? negative + 1 : 0;
                int newNegative = positive + 1;
                positive = newPositive;
                negative = newNegative;
            }
            result = Math.max(result, positive);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#1696 Jump Game VI

Description: From index 0, each move jumps 1 to `k` indices forward; maximize the sum of values landed on, reaching the last index. **Intuition:** `dp[i]` = best score reaching `i` = `nums[i] +` max `dp[j]` in the window `[i-k, i-1]`; a monotonic queue keeps that window max in O(1). **Algorithm:** `dp[i]` over a sliding-window-max queue of indices; answer `dp[n-1]`.

Variables: `dp[i]` = max score reaching index `i`; `queue` = monotonic deque of indices with decreasing `dp` values within the window `[i-k, i-1]`, so its front always holds the window's max. **Pseudocode:**

```
dp[0] = nums[0]; push index 0
for i from 1 to n-1:
    while queue front index < i-k: pop front      // drop out-of-window indices
    dp[i] = nums[i] + dp[queue.front]             // best reachable predecessor
    while queue not empty and dp[queue.back] <= dp[i]: pop back  // keep decreasing
    push i
return dp[n-1]
```

```java
class Solution {
    public int maxResult(int[] nums, int k) {
        int n = nums.length;
        int[] dp = new int[n];
        dp[0] = nums[0];
        Deque<Integer> queue = new ArrayDeque<>();              // ← VARIATION: monotonic queue of indi
        queue.offerLast(0);
        for (int i = 1; i < n; i++) {
            while (!queue.isEmpty() && queue.peekFirst() < i - k) {
                queue.pollFirst();                              // ← VARIATION: drop indices outside wi
            }
            dp[i] = nums[i] + dp[queue.peekFirst()];            // window max is at front
            while (!queue.isEmpty() && dp[queue.peekLast()] <= dp[i]) {
                queue.pollLast();                               // maintain decreasing dp values
            }
            queue.offerLast(i);
        }
        return dp[n-1];
    }
}
```

Time O(n) | **Space** O(n)

#1884 Egg Drop With 2 Eggs and N Floors

Description: With exactly 2 eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case. **Intuition:** `dp[i]` = min worst-case moves for `i` floors; dropping at floor `j` costs `1 + max(dp[j-1] from break, dp[i-j] from survive)`. **Algorithm:** minimize over first-drop floor `j`; answer `dp[n]`.

Variables: `dp[i]` = minimum worst-case moves to find the critical floor among `i` floors with 2 eggs; `j` = floor of the first drop (break -> `j-1` floors below with 1 egg = linear search; survive -> `dp[i-j]` floors above with 2 eggs).
Pseudocode:

```
for i from 1 to n:
    dp[i] = i                       // fallback: 1-egg linear behavior
    for j from 1 to i:              // first drop at floor j
        dp[i] = min(dp[i], 1 + max(j-1, dp[i-j]))   // break vs survive
return dp[n]
```

```java
class Solution {
    public int twoEggDrop(int n) {
        int[] dp = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            dp[i] = i;                                          // worst case with 1 egg behavior
            for (int j = 1; j <= i; j++) {                      // ← VARIATION: first drop at floor j
                dp[i] = Math.min(dp[i], 1 + Math.max(j - 1, dp[i - j]));
            }
        }
        return dp[n];
    }
}
```

Time $O(n^2)$ | **Space** O(n)